PROJECT HANDBOOK

ArchitectPoint

The complete team guide: database, security, JWT, email OTP, every backend file and every Angular component, with diagrams.

Frontend	Angular 16 · port 8081
Backend	Spring Boot 3 · Java 17 · port 8080
Database	MySQL · appdb
Team	Spring Warriors

Contents

1. What ArchitectPoint is

1.1 The three kinds of people

1.2 Technology used

1.3 The big picture

1.4 What is graded

2. Folder structure

2.1 Top level

2.2 Backend packages (com.examly.springapp)

2.3 Frontend folders (angularapp/src/app)

2.4 The layer rule (backend)

3. How the database stores data

3.1 How tables are created (we never write CREATE TABLE)

3.2 Entity-relationship diagram

3.3 Table by table

3.4 Repositories: how we read data without writing SQL

3.5 Resolving "id-only" objects

3.6 Important database rule for tests

4. Security in depth (login, JWT token, roles)

4.1 The security files and their jobs

4.2 How a token is generated (login)

4.3 What happens on every request after login

4.4 SecurityConfig line by line

4.5 The role matrix (who may call what)

4.6 Passwords

4.7 CORS - letting the frontend address call the backend

4.8 Frontend side of security

5. Email OTP verification in depth

5.1 The rules

5.2 The full flow

5.3 OtpService line by line

5.4 How the email is actually sent (SmtpMailer)

5.5 Where the email keys live

5.6 The Otp row through its life

5.7 The frontend part: OtpVerificationComponent

6. Backend reference (every file)

6.1 All API endpoints

6.2 Controller annotations explained

6.3 DTOs (dto package)

6.4 Services - every method

6.5 Errors: from exception to HTTP status

7. Frontend core (Angular)

7.1 What happens when the page opens

7.2 environment.ts - the backend address

7.3 AuthInterceptor - one place that adds the token

7.4 Guards - who may open which page

7.5 Services

7.6 Models (TypeScript interfaces)

7.7 Routes

7.8 Where the pictures come from

7.9 Styling and themes

8. Every component, one by one

8.1 App shell and navigation

8.2 Login, signup, password

8.3 User screens

8.4 Admin screens

8.5 Drawing components (no logic)

9. End-to-end flows

9.1 The life of an inquiry

9.2 Browse → send inquiry

9.3 Follow-up conversation

9.4 Close → feedback → ranking

9.5 Master admin promotes a user

9.6 Soft delete of an architect

10. Integration, running and testing

10.1 How frontend and backend are connected

10.2 Logins

10.3 Running the graded tests

10.4 Troubleshooting

10.5 Glossary

1. What ArchitectPoint is

ArchitectPoint is a web application that connects people who want to build something (a house, an office, a garden) with architects. A user can browse architects, send them an inquiry, ask follow-up questions, and leave star feedback once the work is done. Admins manage the architect list, answer inquiries and read feedback. One special **Master Admin** can promote users to admin or remove admin rights.

1.1 The three kinds of people

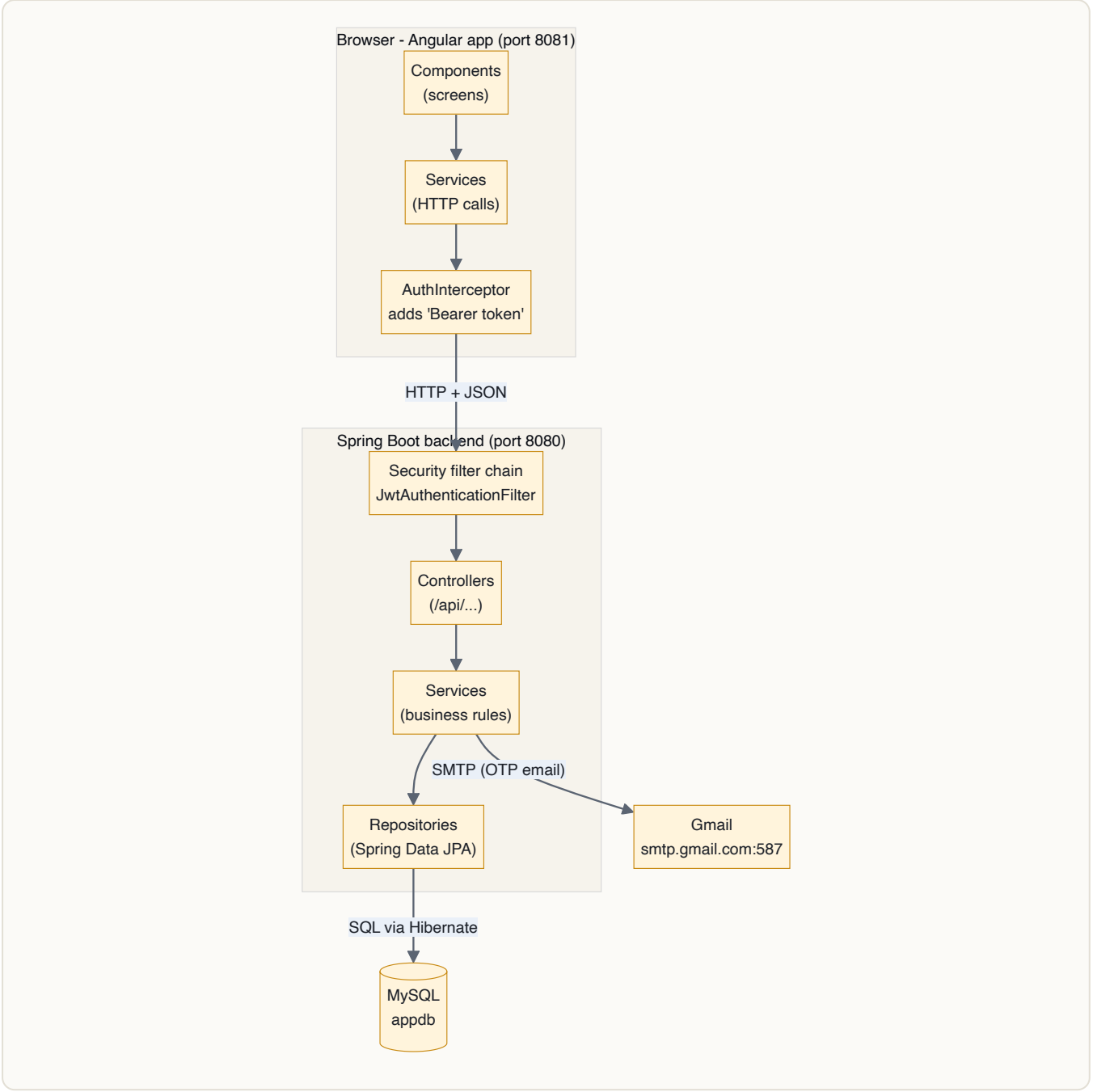
Role	How they get it	What they can do
User	Signs up on the Register page (email must be verified with an OTP).	Browse architects, view portfolios, send inquiries, follow up on their own inquiries, give star feedback after an inquiry is closed, change password.
Admin	Promoted by the Master Admin (or created through the API by the graded tests).	Dashboard, add / edit / hide architects, see all inquiries, change inquiry status, reply to follow-ups, read and sort all feedback.
Master Admin	Built in. Created automatically the first time someone logs in with `admin@architectpoint.com` / `Admin@123`.	Everything an Admin can do, plus the **Admins** page: make a user an admin, or remove admin rights. The Master Admin can never be removed.

1.2 Technology used

Part	Technology	Runs on
Frontend	Angular 16 (TypeScript, HTML, CSS)	Port **8081** (`npm start`)
Backend	Spring Boot 3.0.1, Java 17, Spring Security 6, Spring Data JPA (Hibernate)	Port **8080** (`mvn spring-boot:run`)
Database	MySQL, database name `appdb`	Port 3306
Login tokens	JWT (JSON Web Token) signed with HS256, library `jjwt`	-
Passwords and OTP codes	BCrypt hashing	-
Email	Gmail SMTP (`smtp.gmail.com:587`) through our own small SMTP client	-

1.3 The big picture

Every screen you see is an Angular component running in the browser. When a screen needs data, it calls a **service**, the service sends an HTTP request to the Spring Boot backend, the backend checks the login token, runs the business rules and reads or writes MySQL, then sends JSON back.



> **Key idea:** the frontend never talks to the database directly. It only talks to the backend through `/api/...` URLs. The backend is the only part that knows the database password and the email password.

1.4 What is graded

The project is scored by **24 automated tests**: 15 JUnit tests for the backend (`junit/`) and 9 Karma tests for the frontend (`karma/`). Both currently pass. Some design decisions in this guide exist only to keep those tests passing; they are marked **TEST RULE**.

2. Folder structure

2.1 Top level

```
Gladiator/
├── springapp/          Spring Boot backend (Java)
│   ├── pom.xml         Maven dependencies (do NOT edit - graded)
│   ├── config/         application.properties with the Gmail keys (NOT committed to Git)
│   └── src/main/
│       ├── java/com/examly/springapp/   all Java code (packages below)
│       └── resources/application.properties   main settings
├── angularapp/         Angular frontend
│   └── src/
│       ├── environments/   backend URL lives here
│       ├── app/            all Angular code (folders below)
│       ├── styles.css      global design system (colours, buttons, tables, themes)
│       └── index.html      the single HTML page Angular boots into
├── junit/              the graded backend tests (source of truth)
└── karma/              the graded frontend tests (source of truth)
```

2.2 Backend packages (`com.examly.springapp`)

Package	What lives there	Files
`model`	Entities = Java classes that become database tables (plus two plain response classes)	User, Architect, ArchitectInquiry, SubInquiry, Feedback, Otp, LoginDTO, TopArchitect
`repository`	Interfaces that read/write tables. Spring writes the SQL for us.	UserRepo, ArchitectRepo, ArchitectInquiryRepo, FeedbackRepo, OtpRepo
`service`	Business rules ("an inactive architect cannot get inquiries", "rating must be 1-5"...)	UserService(+Impl), ArchitectService(+Impl), ArchitectInquiryService(+Impl), FeedbackService(+Impl), OtpService
`controller`	REST endpoints. Receive HTTP, call a service, return JSON.	AuthController, ArchitectController, ArchitectInquiryController, FeedbackController, OtpController, AdminController
`dto`	Small "shape" classes (Java records) for request/response bodies that are not database tables	UserResponse, ChangePasswordRequest, SubInquiryRequest, OtpSendRequest, OtpVerifyRequest, OtpSentResponse
`config`	Security (JWT, roles, passwords), CORS, and the email sender	SecurityConfig, JwtUtils, JwtAuthenticationFilter, JwtAuthenticationEntryPoint, MyUserDetailsService, UserPrinciple, CrosConfig, SmtpMailer
`exceptions`	Custom errors and the one place that turns them into HTTP status codes	GlobalExceptionHandler + 6 exception classes

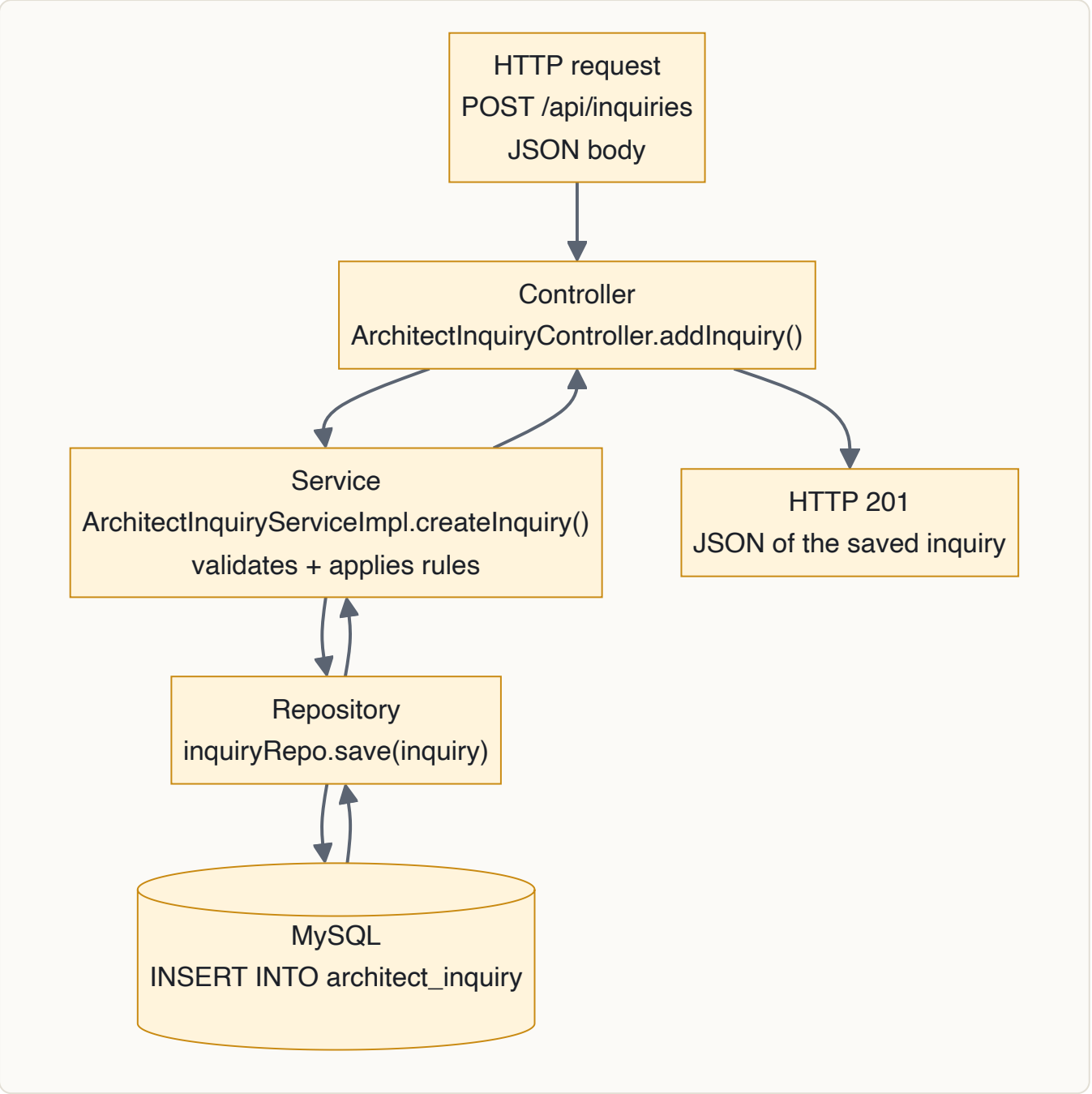
2.3 Frontend folders (`angularapp/src/app`)

Folder / file	What lives there

`components/`	Every screen and every reusable piece (29 components, plus the root `AppComponent` next to this folder). Each has `.ts` (logic), `.html` (template) and `.css` (styles).
`services/`	One class per backend area. They make the HTTP calls: AuthService, ArchitectService, ArchitectInquiryService (file name `architect-inquery.service.ts` - misspelled on purpose, **TEST RULE**), FeedbackService, AdminService.
`models/`	TypeScript interfaces describing the JSON: User, Login, Architect, TopArchitect, ArchitectInquiry, SubInquiry, Feedback.
`guards/auth.guard.ts`	`authGuard` (must be logged in, with the right role) and `guestGuard` (login/signup only when logged out).
`interceptors/auth.interceptor.ts`	Adds the JWT to every backend request; logs out on 401.
`error-message.ts`	`describeError()` - turns any HTTP error into a friendly sentence. Used by every screen.
`app-routing.module.ts`	URL → component table, with guards.
`app.module.ts`	Registers all components, the interceptor, and the startup step that restores the session.
`../environments/environment.ts`	**The backend URL.** The only place you change to point the app at another server.

2.4 The layer rule (backend)

Every backend feature follows the same four steps. If you understand one, you understand all of them.



- **Controller** - only translates HTTP to Java and back. No rules here.
- **Service** - all rules and checks. Throws an exception if something is wrong.
- **Repository** - only database access. We write method *names*, Spring writes the SQL.
- **Entity** - the Java object that is stored as one row.

3. How the database stores data

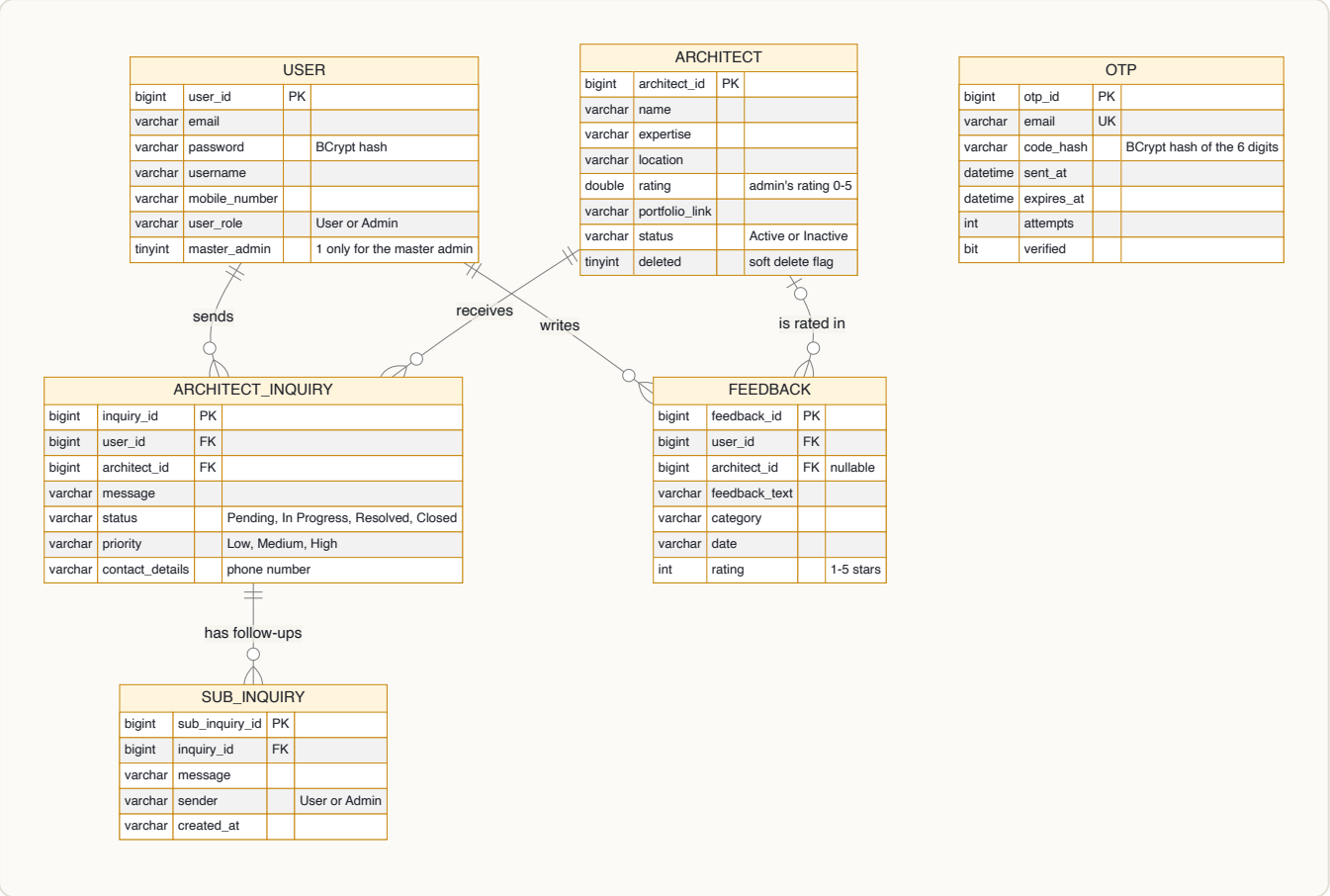
3.1 How tables are created (we never write CREATE TABLE)

We use **JPA / Hibernate**. Every class marked `@Entity` in the `model` package becomes a table. Every field becomes a column. When the backend starts, Hibernate compares the classes with the database and adds anything missing, because of this line in `application.properties`:

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://localhost/appdb?createDatabaseIfNotExist=true
```

- `createDatabaseIfNotExist=true` - MySQL creates the `appdb` database automatically the first time.
- `ddl-auto=update` - Hibernate **adds** new tables/columns but **never deletes** old ones and never deletes rows. That is why data survives a restart.
- Java names are written in camelCase (`mobileNumber`); Hibernate converts them to snake_case columns (`mobile_number`). Class `ArchitectInquiry` becomes table `architect_inquiry`.

3.2 Entity-relationship diagram



The `otp` table is on its own on purpose: it is filled *before* a user exists (during signup), so it is linked by email, not by a foreign key.

3.3 Table by table

user

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;
    private String email;
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private String password;
    private String username;
    private String mobileNumber;
    private String userRole;
    @JsonProperty(access = JsonProperty.Access.READ_ONLY)
    @Column(nullable = false, columnDefinition = "boolean default false")
    private boolean masterAdmin;
}
```

Line	Meaning
`@Id @GeneratedValue(IDENTITY)`	Primary key. MySQL gives 1, 2, 3... automatically (AUTO_INCREMENT).
`@JsonProperty(WRITE_ONLY)` `password`	The password can be **received** in JSON (register) but is **never sent back** in any response.
`@JsonProperty(READ_ONLY)` `masterAdmin`	Can be shown in responses but **cannot be set** from a request. Nobody can make themselves master admin by sending `"masterAdmin": true`.
`columnDefinition = "boolean default false"`	Old rows get `0` when the column is added.

What a stored row looks like (note the password is a hash, not the real password):

```
user_id | email            | password                        | user_role | master_admin
1       | Rohit@gmail.com  | $2a$10$2dch4cD0QjUEH...         | User      | 0
6       | admin@architect..| $2a$10$...                      | Admin     | 1
```

architect

Fields: `architectId, name, expertise, location, rating, portfolioLink, status` and a hidden `deleted` flag.

- **Soft delete:** deleting an architect only sets `deleted = 1`. The row stays, so old inquiries and feedback still show the architect's name. Lists use `findByDeletedFalse()` so hidden architects disappear from screens.
- `@JsonIgnore deleted` - the flag is never sent to the frontend.
- `status = "Inactive"` means "temporarily unavailable" - the architect stays visible but cannot receive new inquiries.

architect_inquiry and sub_inquiry

```
@ManyToOne
@JoinColumn(name = "userId", nullable = false)
private User user;

@ManyToOne
@JoinColumn(name = "architectId", nullable = false)
private Architect architect;

@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.EAGER)
@JoinColumn(name = "inquiryId")
@OrderBy("subInquiryId")
private List<SubInquiry> subInquiries = new ArrayList<>();
```

Annotation	Meaning
`@ManyToOne user / architect`	Many inquiries belong to one user and one architect. Stored as foreign-key columns `user_id` and `architect_id`. In JSON the whole user/architect object is included.
`@OneToMany ... subInquiries`	One inquiry has many follow-up messages, stored in table `sub_inquiry` with column `inquiry_id`.
`cascade = ALL`	Saving the inquiry also saves new follow-ups. Deleting the inquiry deletes its follow-ups.
`orphanRemoval = true`	A follow-up removed from the list is deleted from the table.
`fetch = EAGER` + `@OrderBy`	Follow-ups are always loaded with the inquiry, oldest first.

> Relations are **one-directional** (an inquiry knows its user, but `User` has no list of inquiries). If both sides pointed at each other, turning them into JSON would loop forever.

feedback

Fields: `feedbackId, feedbackText, date, category, rating (1–5)`, the `user` (required) and the `architect` (optional, `nullable = true`). The star rating is what feeds the admin "user rating" average and the home page Top 3.

otp

One row per email address that requested a code (`email` is UNIQUE). Sending a new code **reuses** the same row and resets it. The real 6-digit code is never stored, only its BCrypt hash. See chapter 5 for the full flow.

3.4 Repositories: how we read data without writing SQL

A repository is an interface that extends `JpaRepository<Entity, IdType>`. That alone gives us `save()`, `findById()`, `findAll()`, `delete()`. Extra queries are written as **method names**; Spring reads the name and builds the SQL.

Repository method	SQL Spring generates (simplified)

`UserRepo.findByEmail(email)`	`SELECT * FROM user WHERE email = ?`
`ArchitectRepo.findByDeletedFalse()`	`SELECT * FROM architect WHERE deleted = false`
`ArchitectInquiryRepo.findByUserUserId(id)`	`SELECT * FROM architect_inquiry WHERE user_id = ?` (read as "find by user → userId")
`ArchitectInquiryRepo.findByArchitectArchitectId(id)`	`... WHERE architect_id = ?`
`FeedbackRepo.findByUserUserId(id)`	`SELECT * FROM feedback WHERE user_id = ?`
`FeedbackRepo.findByArchitectArchitectId(id)`	`... WHERE architect_id = ?`
`OtpRepo.findByEmail(email)`	`SELECT * FROM otp WHERE email = ?`

3.5 Resolving "id-only" objects

The frontend sends related objects as just an id, for example `{"user": {"userId": 2}, "architect": {"architectId": 1}}`. Before saving, the service loads the real rows, otherwise the insert would fail:

```
inquiry.setUser(userRepo.findById(userId)
        .orElseThrow(() -> new ArchitectInquiryException("User with id " + userId + " does not
exist.")));
inquiry.setArchitect(architectRepo.findById(architectId)
        .orElseThrow(() -> new ArchitectInquiryException("Architect with id " + architectId + "
does not exist.")));
```

3.6 Important database rule for tests

TEST RULE The JUnit tests expect brand-new ids (user 1 and 2, architect 1, inquiry 1). Because `ddl-auto=update` keeps old rows, always reset the database before running the backend tests:

```
mysql -u root -p -e "DROP DATABASE IF EXISTS appdb; CREATE DATABASE appdb;"
```

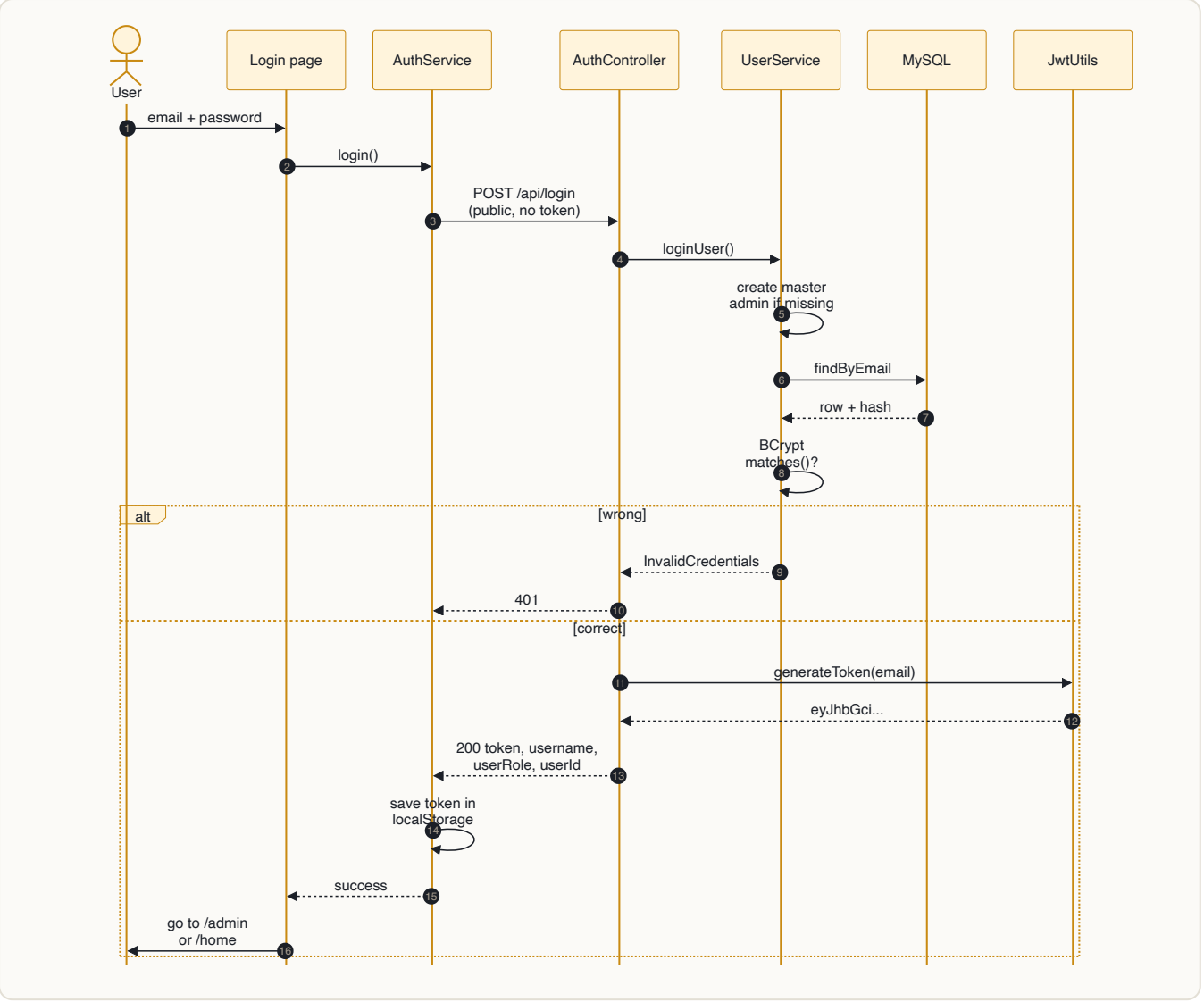
4. Security in depth (login, JWT token, roles)

Security answers two questions on every request: **"Who are you?"** (authentication) and **"Are you allowed to do this?"** (authorization). We use a **JWT** - a signed text token the backend gives you at login. You send it back with every request; the backend checks the signature to know it is genuine.

4.1 The security files and their jobs

File (config package)	Job in one line
`SecurityConfig`	The rule book: which URLs are public, which need a login, which need a role. Also defines BCrypt.
`JwtUtils`	Makes a token at login, reads the email out of a token, checks a token is valid.
`JwtAuthenticationFilter`	Runs on every request. Reads the `Authorization: Bearer ...` header and tells Spring who the user is.
`MyUserDetailsService`	Loads a user from the database by email, for Spring Security.
`UserPrinciple`	Wraps our `User` so Spring understands it, and turns `userRole` into Spring "authorities" (ROLE_User, ROLE_Admin, ROLE_MasterAdmin).
`JwtAuthenticationEntryPoint`	What to answer when someone is not logged in: HTTP **401 Unauthorized**.
`CrosConfig`	CORS: which frontend addresses may call the backend from a browser.

4.2 How a token is generated (login)



JwtUtils line by line

```
private static final String SECRET = "architectPointSecretKeyForJwtSigning2025";
private static final long EXPIRATION_MS = 24 * 60 * 60 * 1000L;

public String generateToken(String email) {
    return Jwts.builder()
            .setSubject(email)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))
            .signWith(SignatureAlgorithm.HS256, SECRET)
            .compact();
}

public String extractUsername(String token) {
    return Jwts.parser().setSigningKey(SECRET).parseClaimsJws(token).getBody().getSubject();
}

public boolean validateToken(String token) {
    try {
        Jwts.parser().setSigningKey(SECRET).parseClaimsJws(token);
        return true;
    } catch (Exception e) {
        return false;
    }
}
```

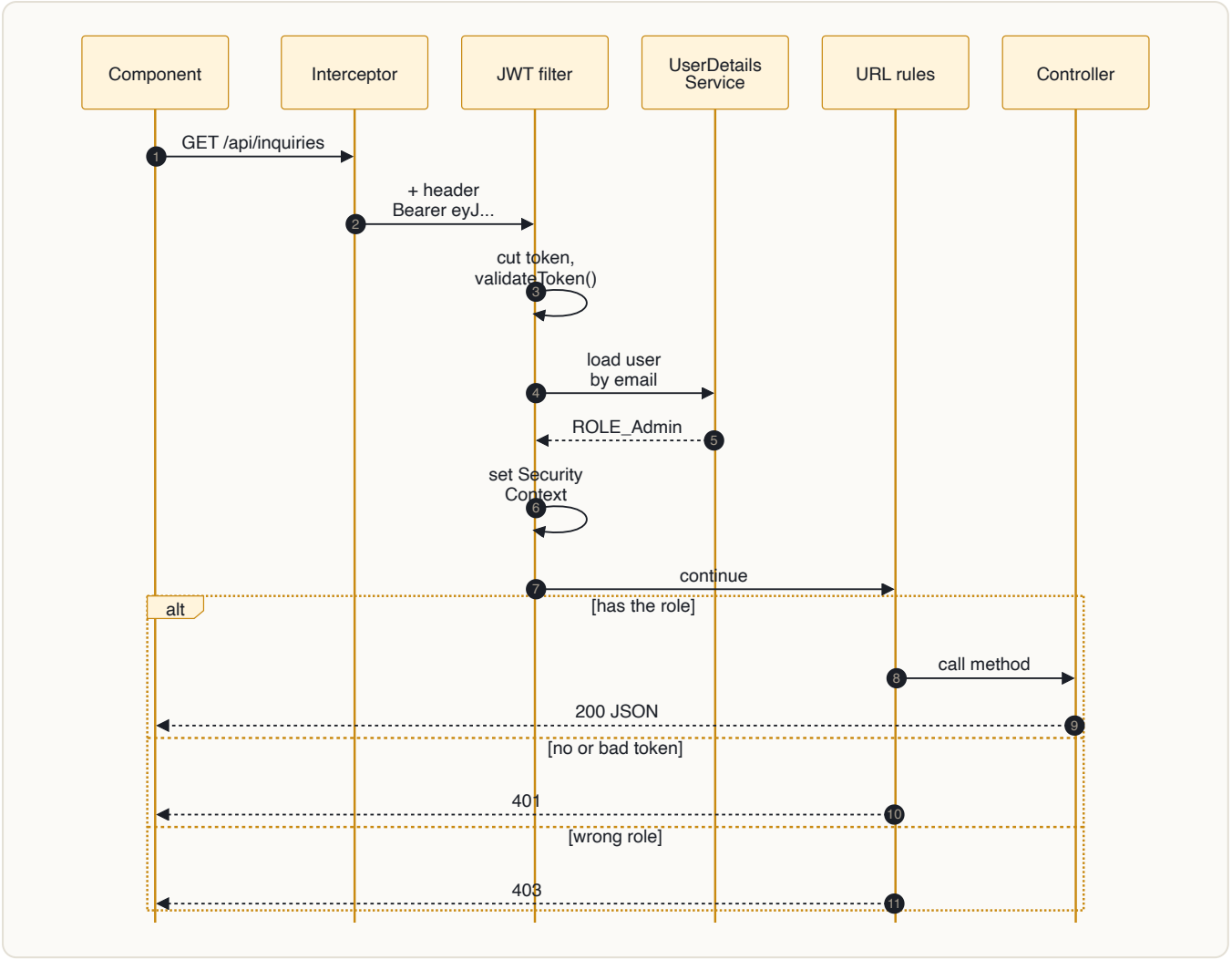
Line	What it does
`SECRET`	The private key used to sign tokens. Only the backend knows it. Anyone changing a token without it breaks the signature.
`EXPIRATION_MS`	24 hours. After that the token is rejected and the user must log in again.
`setSubject(email)`	The **only** personal data inside the token is the email. No role, no id, no password.
`setIssuedAt` / `setExpiration`	Stores "made at" and "valid until" timestamps inside the token.
`signWith(HS256, SECRET)`	Computes an HMAC-SHA256 signature over the token using the secret.
`compact()`	Produces the final string `header.payload.signature`.
`extractUsername`	Verifies the signature and returns the email (the "subject").
`validateToken`	`true` if the signature is correct and the token is not expired; any problem → `false`.

A real token decoded (the middle part is only Base64, not encrypted - never put secrets in it):

```
header    {"alg":"HS256"}
payload   {"sub":"seed.user@example.com","iat":1791403940,"exp":1791490340}
signature HMAC-SHA256(header + "." + payload, SECRET)
```

> **Why the role is not inside the token:** the role is read fresh from the database on every request. If the Master Admin removes someone's admin rights, it takes effect immediately - the old token cannot keep admin power.

4.3 What happens on every request after login



JwtAuthenticationFilter line by line

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Autowired private JwtUtils jwtUtils;
    @Autowired private MyUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException,
IOException {
        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            try {
                if (jwtUtils.validateToken(token)) {
                    String email = jwtUtils.extractUsername(token);
                    if (email != null && SecurityContextHolder.getContext().getAuthentication()
== null) {
                        UserDetails userDetails = userDetailsService.loadUserByUsername(email);
                        UsernamePasswordAuthenticationToken auth =
                                new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
                        auth.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
                        SecurityContextHolder.getContext().setAuthentication(auth);
                    }
                }
            } catch (Exception e) {
                SecurityContextHolder.clearContext();
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Step	Explanation
`extends OncePerRequestFilter`	Spring guarantees this runs exactly once for every HTTP request.
`getHeader("Authorization")`	Reads the header the Angular interceptor added.
`startsWith("Bearer ")` / `substring(7)`	The header looks like `Bearer eyJ...`; we remove the first 7 characters ("Bearer ") to get the token.
`validateToken` → `extractUsername`	Signature and expiry are checked; then the email is read.
`getAuthentication() == null`	Only set the user if nobody is set yet for this request.
`loadUserByUsername(email)`	Loads the user row from MySQL (so the role is always current).
`new UsernamePasswordAuthenticationToken(...)`	Spring's "this person is logged in" object, including their authorities (roles).
`SecurityContextHolder...setAuthentication(auth)`	Saves "who is calling" for the rest of this request. Controllers can then receive `Principal` / `Authentication`.

`catch → clearContext()`	Any broken token = treated as not logged in.
`filterChain.doFilter(...)`	Always pass the request on. If no user was set, the URL rules will answer 401 or 403.

UserPrinciple - turning roles into authorities

```
public Collection<? extends GrantedAuthority> getAuthorities() {
    List<GrantedAuthority> authorities = new ArrayList<>();
    authorities.add(new SimpleGrantedAuthority("ROLE_" + user.getUserRole()));
    if (user.isMasterAdmin()) {
        authorities.add(new SimpleGrantedAuthority("ROLE_MasterAdmin"));
    }
    return authorities;
}
public String getUsername() { return user.getEmail(); }
```

A normal user gets `[ROLE_User]`, an admin `[ROLE_Admin]`, the master admin `[ROLE_Admin, ROLE_MasterAdmin]`. In `SecurityConfig`, `hasRole("Admin")` means "has authority `ROLE_Admin`". The "username" for Spring is our email.

4.4 SecurityConfig line by line

```
@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable())
      .cors(cors -> {})
      .exceptionHandling(e -> e.authenticationEntryPoint(jwtAuthenticationEntryPoint))
      .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/api/register", "/api/login", "/api/otp/**").permitAll()
          .requestMatchers("/error").permitAll()
          .requestMatchers("/api/admin/users/**").hasRole("MasterAdmin")
          ... (one line per endpoint, table below)
          .anyRequest().authenticated())
      .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

Setting	Why
`BCryptPasswordEncoder`	Hashes passwords (and OTP codes). A hash cannot be turned back into the password; at login we compare with `matches()`. Each hash includes a random "salt", so the same password gives a different hash each time.
`csrf.disable()`	CSRF attacks use browser cookies. We use a header token, not cookies, so CSRF protection is not needed.
`cors(cors -> {})`	Turns on CORS using the rules in `CrosConfig`.

`authenticationEntryPoint`	When a protected URL is called without a valid token, answer **401**.
`STATELESS`	The server keeps no login session. Every request must carry its token.
`permitAll()`	Public URLs: register, login, OTP send/verify, and Spring's error page.
`"/error" permitAll`	Without it a 403 would be turned into a 401 when Spring forwards to its error page.
`requestMatchers(METHOD, "/api/x/*")`	Rules are checked **top to bottom; the first match wins**. That is why more specific URLs (like `/api/inquiries/user/*`) come before broader ones. We use a single `*` (one path segment), never `**`.
`anyRequest().authenticated()`	Anything not listed still needs a valid token.
`addFilterBefore(jwtAuthenticationFilter, ...)`	Our JWT filter runs before Spring's own login filter, so the user is known in time.

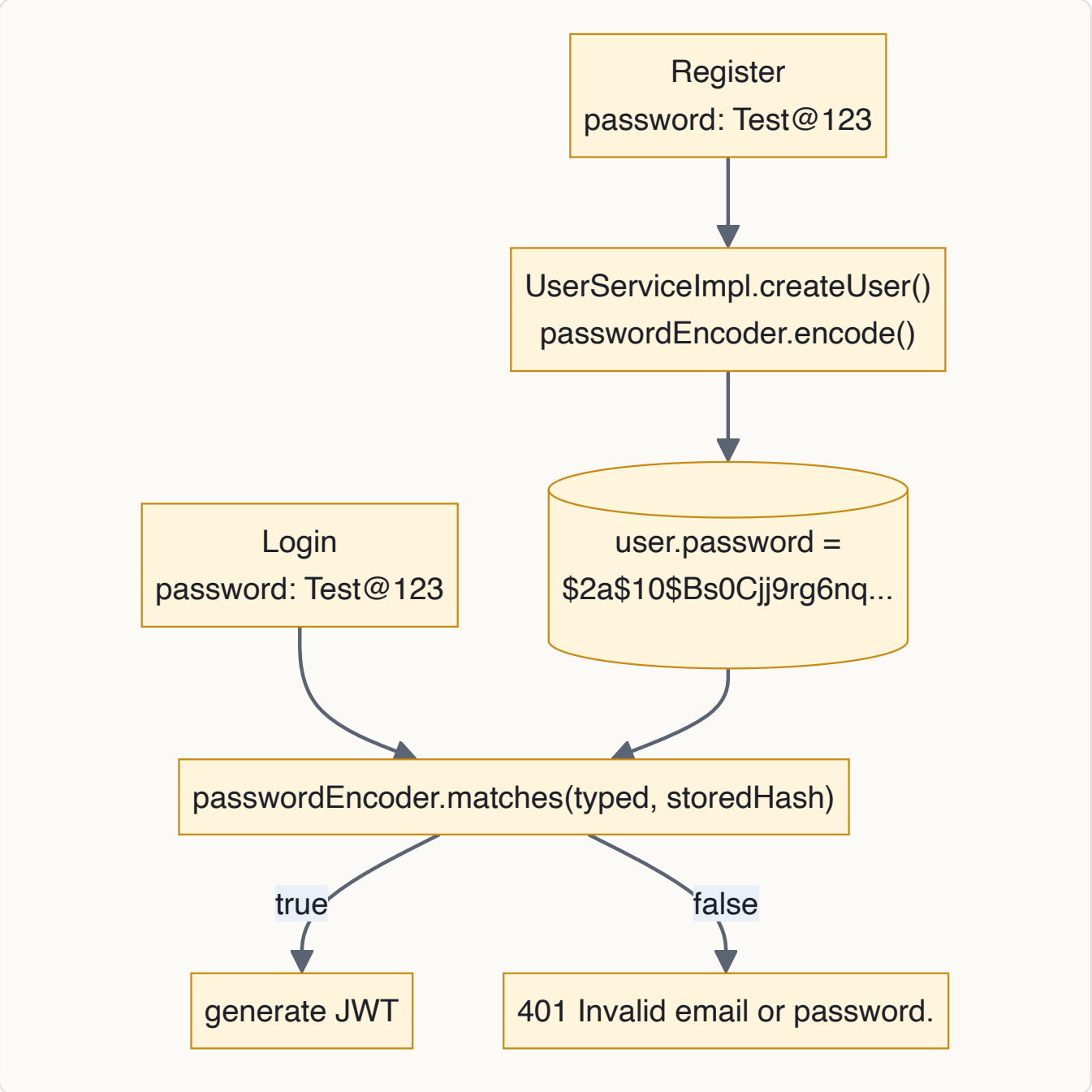
4.5 The role matrix (who may call what)

Endpoint	Method	Rule	User	Admin
/api/register, /api/login, /api/otp/send, /api/otp/verify	POST	public	yes	yes
/api/me, /api/me/password	GET, PUT	logged in	yes	yes
/api/architects	GET	logged in	yes	yes
/api/architects/top-rated	GET	logged in	yes	yes
/api/architects	POST	Admin	403	yes
/api/architects/{id}	GET, PUT, DELETE	Admin	403	yes
/api/inquiries	POST	User	yes	403
/api/inquiries	GET	Admin	403	yes
/api/inquiries/user/{userId}	GET	User	yes	403
/api/inquiries/{id}	PUT	Admin	403	yes
/api/inquiries/{id}	DELETE	User	yes	403
/api/inquiries/{id}/subinquiries	POST	logged in (+ owner check in service)	own only	yes
/api/feedback	POST	User	yes	403
/api/feedback	GET	Admin	403	yes
/api/feedback/user/{userId}	GET	User	yes	403
/api/feedback/{id}	DELETE	User	yes	403

/api/admin/users/**	GET, PUT	MasterAdmin	403	403 (master only)

TEST RULE 401 means "no valid token". 403 means "valid token, wrong role". The graded tests check both.

4.6 Passwords



- **Change password** (`PUT /api/me/password`): the service checks the current password with `matches()`, requires at least 6 characters, refuses the same password, then saves the new hash.
- **Master Admin**: `createMasterAdminIfMissing()` runs during login. If someone logs in with `admin@architectpoint.com` and that row does not exist yet, it is created with the hashed password `Admin@123`, role Admin and `masterAdmin = true`. It is created at login rather than at startup so that brand-new databases still give ids 1 and 2 to the graded test users. **TEST RULE**
- Nobody can register with the master admin's email (`"This email is reserved."`).

4.7 CORS - letting the frontend address call the backend

Browsers block a page on one address (e.g. port 8081) from calling another address (port 8080) unless the server says it is allowed. That permission is CORS.

```
@Value("${app.frontend-url:*}")
private String frontendUrl;

public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/**")
            .allowedOriginPatterns(frontendUrl.split(","))
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*");
}
```

`app.frontend-url` defaults to `*` (any address), which is safe here because we do not use cookies. To lock it to your frontend, set `app.frontend-url=https://8081-xxxx...` in `application.properties` (comma-separate several).

4.8 Frontend side of security

The backend is the real protection. The frontend adds a friendly layer so users never see broken pages:

- **AuthInterceptor** - attaches the token to every backend call and logs the user out on a 401 (expired token). See 7.3.
- **authGuard** - blocks routes when logged out (→ /login) or when the role is wrong (→ "403 Access denied" page). See 7.4.
- **Only the token is stored** in `localStorage`. The user's name, id and role are fetched from `GET /api/me` when the app starts and kept in memory.

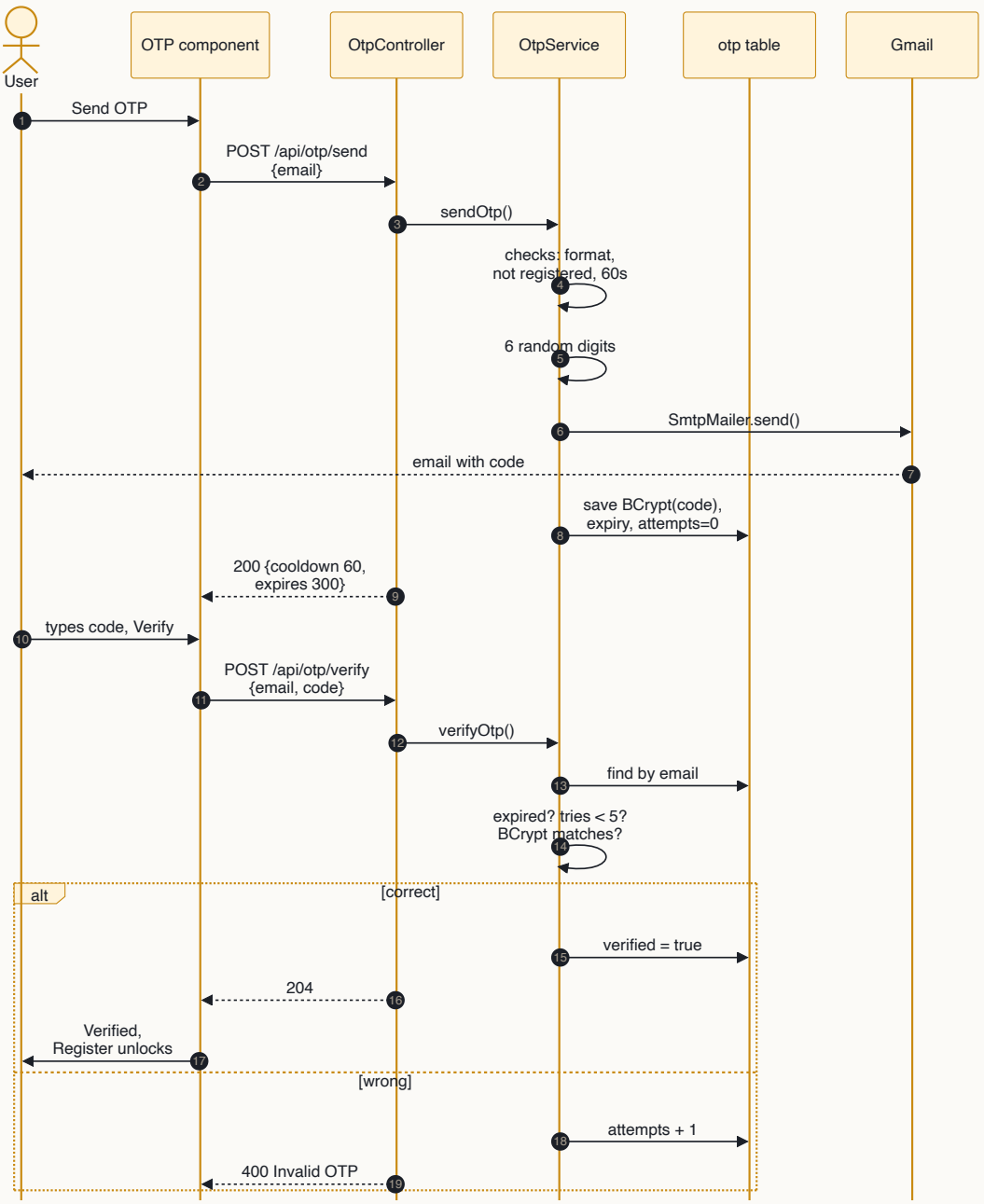
5. Email OTP verification in depth

When someone signs up, we must be sure the email really belongs to them. So before the Register button unlocks, we email a **one-time password (OTP)**: a random 6-digit code. The user types it back; if it matches, the email is verified.

5.1 The rules

Rule	Value	Where it is set
Code length	6 digits (000000-999999)	`OtpService`
Code expires after	5 minutes	`app.otp.expiry-minutes=5`
Wait before "Resend OTP"	60 seconds	`app.otp.resend-cooldown-seconds=60`
Wrong tries allowed	5, then a new code is needed	`app.otp.max-attempts=5`
Show code on screen when no mail server (testing only)	off	`app.otp.show-undelivered-code=${OTP_DEV_MODE:false}`
Already registered email	refused	`OtpService.sendOtp()`

5.2 The full flow



5.3 OtpService line by line

sendOtp(email)

```
public OtpSentResponse sendOtp(String rawEmail) {
    String email = normalize(rawEmail);
    if (!email.matches(EMAIL_PATTERN)) {
        throw new InvalidRequestException("Please enter a valid email");
    }
    if (userRepo.findByEmail(email).isPresent()) {
        throw new InvalidRequestException("This email is already registered");
    }
    Otp otp = otpRepo.findByEmail(email).orElseGet(() -> new Otp(email));
    if (otp.getSentAt() != null) {
        long waited = Duration.between(otp.getSentAt(), LocalDateTime.now()).getSeconds();
        if (waited < cooldownSeconds) {
            throw new InvalidRequestException("Please wait " + (cooldownSeconds - waited) + "
seconds before requesting a new OTP");
        }
    }
    String code = String.format("%06d", RANDOM.nextInt(1_000_000));
    boolean delivered = deliver(email, code);
    LocalDateTime now = LocalDateTime.now();
    otp.reset(passwordEncoder.encode(code), now, now.plusMinutes(expiryMinutes));
    otpRepo.save(otp);
    return new OtpSentResponse(cooldownSeconds, expiryMinutes * 60,
            !delivered && showUndeliveredCode ? code : null);
}
```

Line	Explanation
`normalize(rawEmail)`	Trims spaces and lower-cases, so `Ravi@Gmail.com` and `ravi@gmail.com` are the same.
`EMAIL_PATTERN`	Regular expression `^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$` - "something @ something . at-least-2-letters".
`userRepo.findByEmail(...).isPresent()`	Do not send codes to emails that already have an account.
`findByEmail(...).orElseGet(new Otp(email))`	Reuse this email's existing row, or create a new one.
cooldown check	If the last code was sent less than 60 seconds ago, refuse (prevents spamming someone's inbox).
`RANDOM.nextInt(1_000_000)`	`SecureRandom` = cryptographically strong random number 0-999999. `%06d` pads it to 6 digits (e.g. `004215`).
`deliver(email, code)`	Sends the email (5.4). If sending fails, an error is thrown and **nothing is saved**.
`otp.reset(encode(code), now, now+5min)`	Stores the **hash** of the code, the send time, the expiry, and resets attempts to 0 and verified to false.
`new OtpSentResponse(...)`	Tells the screen how long to wait before resend (60) and how long the code lives (300 s). `devCode` is always `null` in real mode.

verifyOtp(email, code)

```
public void verifyOtp(String rawEmail, String code) {
    Otp otp = otpRepo.findByEmail(normalize(rawEmail))
            .orElseThrow(() -> new InvalidRequestException("No OTP was requested for this
email"));
    if (otp.isVerified()) {
        return;
    }
    if (LocalDateTime.now().isAfter(otp.getExpiresAt())) {
        throw new InvalidRequestException("OTP has expired. Please request a new one");
    }
    if (otp.getAttempts() >= maxAttempts) {
        throw new InvalidRequestException("Too many wrong attempts. Please request a new OTP");
    }
    if (code == null || !passwordEncoder.matches(code.trim(), otp.getCodeHash())) {
        otp.recordFailedAttempt();
        otpRepo.save(otp);
        throw new InvalidRequestException("Invalid OTP");
    }
    otp.markVerified();
    otpRepo.save(otp);
}
```

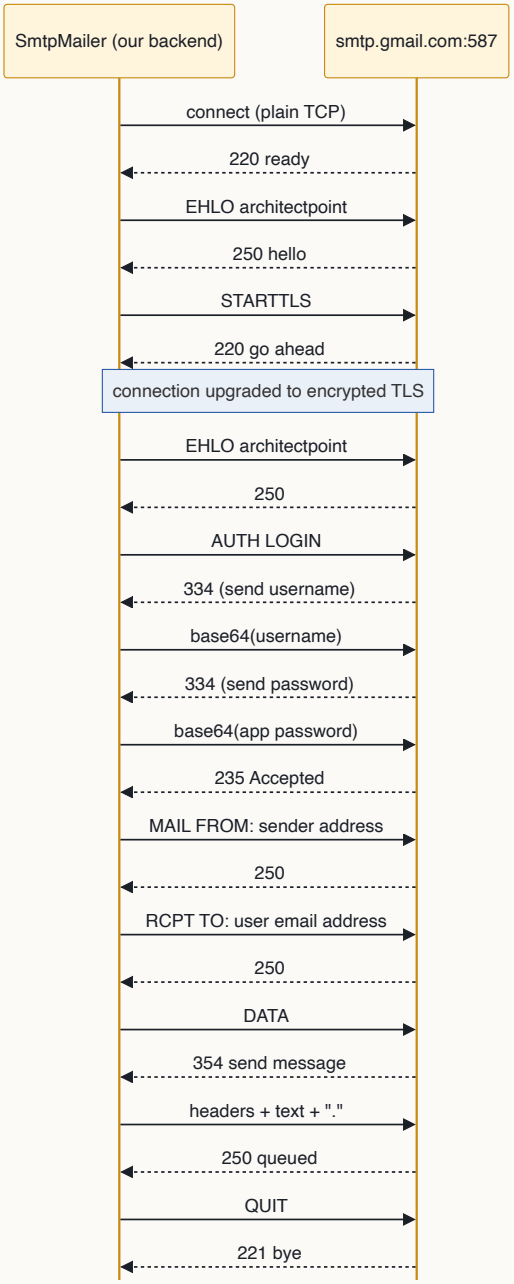
The checks run in order: a code was requested → not already verified → not expired → fewer than 5 wrong tries → the code matches the stored hash. A wrong code increases `attempts`; after 5 the user must click "Resend OTP".

deliver(email, code)

```
private boolean deliver(String email, String code) {
    if (!mailer.isConfigured()) {
        if (showUndeliveredCode) {
            log.warn("[DEV] No mail server configured (MAIL_HOST). OTP for {} is {}", email,
code);
            return false;
        }
        log.error("Cannot send OTP: no mail server configured (set MAIL_HOST, MAIL_USERNAME,
MAIL_PASSWORD)");
        throw new InvalidRequestException("Could not send the OTP right now. Please try again
later");
    }
    try {
        mailer.send(email, "Your ArchitectPoint verification code",
                "Your ArchitectPoint verification code is " + code + ". It expires in " +
expiryMinutes
                        + " minutes. If you did not request it, ignore this email.");
        return true;
    } catch (Exception exception) {
        log.error("Could not email the OTP to {}: {}", email, exception.getMessage());
        throw new InvalidRequestException("Could not send the OTP right now. Please try again
later");
    }
}
```

5.4 How the email is actually sent (SmtpMailer)

Email is sent with the **SMTP** protocol. Our project cannot add Spring's mail library (`pom.xml` must not change, **TEST RULE**), so `SmtpMailer` speaks SMTP directly using only standard Java sockets. It is a short, readable conversation with Gmail:



Piece of SmtpMailer	Explanation
Constructor `@Value("${spring.mail.host:}")` etc.	Reads host, port, username, password and sender from the settings (5.5).
`isConfigured()`	`true` when a mail host is set.
`port == 465 ? tls.createSocket() : new Socket()`	Port 465 = encrypted from the first byte. Port 587 (Gmail default) = start plain, then `STARTTLS` upgrades it.

`session.command(text, expectedCode)`	Sends one line, reads the server's reply, and fails if the reply code is not the expected one (e.g. 250 = OK).
`expect()` loop on `'-'`	Servers can answer with several lines like `250-...`; the last line has a space (`250 ...`). We read until the last line.
`base64(...)`	`AUTH LOGIN` requires username and password in Base64. They travel inside the TLS encryption.
`TIMEOUT_MS = 10000`	Give up after 10 seconds instead of hanging.
`finally { socket.close(); }`	Always close the connection.

5.5 Where the email keys live

The main `application.properties` only reads environment variables (same names as the reference project), with empty defaults:

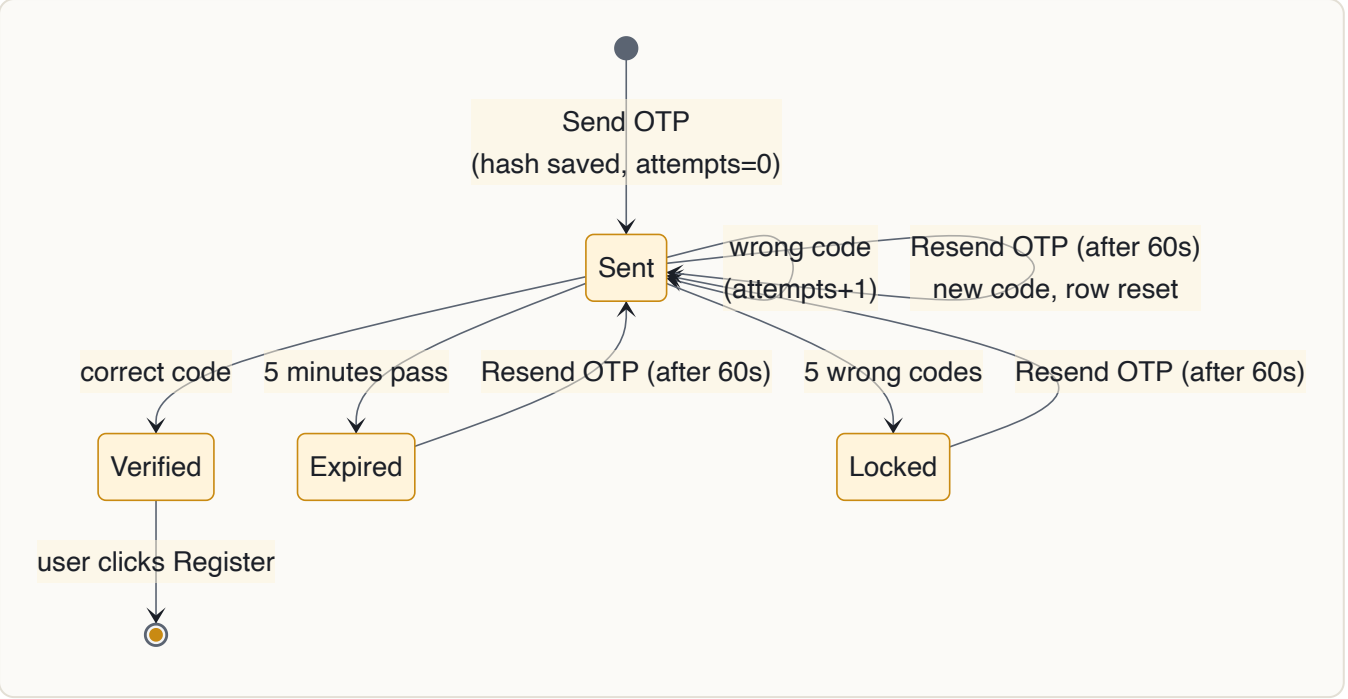
```
spring.mail.host=${MAIL_HOST:}
spring.mail.port=${MAIL_PORT:587}
spring.mail.username=${MAIL_USERNAME:}
spring.mail.password=${MAIL_PASSWORD:}
app.mail.from=${MAIL_FROM:${MAIL_USERNAME:no-reply@architectpoint.com}}
```

The real values are in `springapp/config/application.properties`. Spring Boot automatically loads a `config/application.properties` from the folder you start the backend in, and its values win. That file is listed in `.gitignore`, so the Gmail app password never goes to GitHub.

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=<the Gmail address>
spring.mail.password=<16-character Gmail app password>
app.mail.from=<the Gmail address>
```

> A Gmail **app password** is not the normal Gmail password. Create it at Google Account → Security → 2-Step Verification → App passwords. On another machine or portal, copy this file or set the `MAIL_*` environment variables.

5.6 The Otp row through its life



5.7 The frontend part: OtpVerificationComponent

Used inside the signup form, next to the email box: `<app-otp-verification [email]="user.email" [emailValid]="!!email.valid" (verified)="emailVerified = true">`

Member	Purpose
`@Input() email`, `@Input() emailValid`	The email typed in the form and whether its format is valid (the Send button is disabled until it is).
`@Output() verified`	Fires once the code is accepted. The signup page then sets `emailVerified = true`, locks the email box and enables Register.
`sendOtp()`	Calls `AuthService.sendOtp`, shows the code panel and starts the 60-second countdown.
`verifyOtp()`	Calls `AuthService.verifyOtp`; on success shows "✓ Verified" and emits `verified`.
`startTimer()` / `secondsLeft`	`setInterval` counts down every second; "Resend OTP in 59s" becomes a "Resend OTP" link at 0.
`ngOnChanges()`	If the user edits the email after sending, the panel resets (a code belongs to one email).
`ngOnDestroy()`	Stops the timer when leaving the page.

TEST RULE The register API itself does not demand a verified OTP, because the graded backend tests register users directly without one. The verification is enforced by the signup screen.

6. Backend reference (every file)

6.1 All API endpoints

Method + URL	Controller method	Request body	Response
POST /api/register	AuthController.register	User JSON (username, email, password, mobileNumber, userRole)	201 + saved User (no password)
POST /api/login	AuthController.login	{email, password}	200 LoginDTO {token, username, userRole, userId, masterAdmin}
GET /api/me	AuthController.me	-	UserResponse of the caller
PUT /api/me/password	AuthController.changePassword	ChangePasswordRequest	204
POST /api/otp/send	OtpController.send	OtpSendRequest {email}	OtpSentResponse
POST /api/otp/verify	OtpController.verify	OtpVerifyRequest {email, code}	204
GET /api/architects	ArchitectController.getAllArchitects	-	list (soft-deleted hidden)
GET /api/architects/top-rated	ArchitectController.getTopRated	-	3 × TopArchitect
GET /api/architects/{id}	ArchitectController.getArchitectById	-	Architect
POST /api/architects	ArchitectController.addArchitect	Architect	201 Architect
PUT /api/architects/{id}	ArchitectController.updateArchitect	Architect	Architect
DELETE /api/architects/{id}	ArchitectController.deleteArchitect	-	200 (soft delete)
POST /api/inquiries	ArchitectInquiryController.addInquiry	ArchitectInquiry (user/architect as ids)	201 ArchitectInquiry
GET /api/inquiries	…getAllInquiries	-	all inquiries
GET /api/inquiries/user/{userId}	…getInquiriesByUserId	-	that user's inquiries
PUT /api/inquiries/{id}	…updateInquiry	ArchitectInquiry	updated inquiry
DELETE /api/inquiries/{id}	…deleteInquiry	-	200
POST /api/inquiries/{id}/subinquiries	…addSubInquiry	SubInquiryRequest {message}	201 inquiry with follow-ups
POST /api/feedback	FeedbackController.addFeedback	Feedback	201 Feedback
GET /api/feedback	…getAllFeedback	-	all feedback
GET /api/feedback/user/{userId}	…getFeedbackByUserId	-	that user's feedback
GET /api/feedback/{id}	…getFeedbackById	-	Feedback

DELETE /api/feedback/{id}	…deleteFeedback	-	200
GET /api/admin/users	AdminController.getAllUsers	-	list of UserResponse
PUT /api/admin/users/{id}/make-admin	AdminController.makeAdmin	-	UserResponse
PUT /api/admin/users/{id}/remove-admin	AdminController.removeAdmin	-	UserResponse

6.2 Controller annotations explained

```
@RestController
@RequestMapping("/api/inquiries")
public class ArchitectInquiryController {
    @Autowired
    private ArchitectInquiryService inquiryService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ArchitectInquiry addInquiry(@RequestBody ArchitectInquiry inquiry) {
        return inquiryService.createInquiry(inquiry);
    }

    @GetMapping("/user/{userId}")
    public List<ArchitectInquiry> getInquiriesByUserId(@PathVariable Long userId) { ... }

    @PostMapping("/{inquiryId}/subinquiries")
    @ResponseStatus(HttpStatus.CREATED)
    public ArchitectInquiry addSubInquiry(@PathVariable Long inquiryId,
                                          @RequestBody SubInquiryRequest request,
                                          Authentication auth) {
        boolean fromAdmin = auth.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_Admin"));
        return inquiryService.addSubInquiry(inquiryId, request.message(), auth.getName(),
fromAdmin);
    }
}
```

Annotation	Meaning
`@RestController`	Every method's return value is turned into JSON automatically.
`@RequestMapping("/api/inquiries")`	Common URL prefix for all methods in the class.
`@Autowired`	Spring creates the service once and "injects" it here (dependency injection).
`@GetMapping / @PostMapping / @PutMapping / @DeleteMapping`	Which HTTP method + URL calls this Java method.
`@RequestBody`	Convert the JSON body into this Java object.
`@PathVariable`	Take `{userId}` from the URL.

`@ResponseStatus(CREATED)`	Answer 201 instead of 200.
`Authentication auth` / `Principal principal`	Spring passes in "who is calling" (set by the JWT filter). `getName()` is the email.

6.3 DTOs (dto package)

A DTO (Data Transfer Object) is the shape of a request or response that is not a database table. We use Java **records**: one line declares the fields, Java writes the constructor and getters.

DTO	Fields	Used by
UserResponse	userId, email, username, mobileNumber, userRole, masterAdmin + `from(User)`	/api/me and the Admins page - never exposes the password
ChangePasswordRequest	currentPassword, newPassword	PUT /api/me/password
SubInquiryRequest	message	follow-up messages
OtpSendRequest	email	POST /api/otp/send
OtpVerifyRequest	email, code	POST /api/otp/verify
OtpSentResponse	cooldownSeconds, expiresInSeconds, devCode	answer of /api/otp/send
LoginDTO (model package)	token, username, userRole, userId, masterAdmin	answer of /api/login. Name and fields are fixed by the graded tests **TEST RULE**
TopArchitect (model package)	architect, score, userAverage, reviews	answer of /api/architects/top-rated

6.4 Services - every method

UserServiceImpl

Method	What it does
createUser(user)	Refuses the master admin's email, hashes the password with BCrypt, saves.
loginUser(user)	Creates the master admin if this is its first login, finds the user by email, checks the password; wrong → `InvalidCredentialsException` (401).
getUserByEmail(email)	Used by /api/me and change password.
changePassword(email, current, new)	Checks current password, length ≥ 6, not the same, saves new hash.
getAllUsers()	All users (Admins page).
makeAdmin(id)	Sets role to Admin (error if already admin).
removeAdmin(id)	Sets role back to User. Refuses for the master admin.
createMasterAdminIfMissing(email) *(private)*	See 4.6.

ArchitectServiceImpl

Method	What it does
addArchitect	Saves a new architect.
updateArchitect(id, data)	Loads the existing row, copies the 6 editable fields, saves.
getArchitectById(id)	Or throws `ArchitectException` (404).
getAllArchitects()	`findByDeletedFalse()` - hides soft-deleted ones.
deleteArchitect(id)	**Soft delete**: sets `deleted = true`.
getTopRated(limit)	Active architects only, scored, sorted high → low, first 3.
score(architect) *(private)*	The Top-3 formula below.

```
score = (adminRating × 2 + sum of all user feedback stars) ÷ (2 + number of feedback reviews)
```

The admin's rating counts like two votes; every user review is one vote. Example: admin rating 4.8, one review of 5 stars → (9.6 + 5) ÷ 3 = **4.87**. If that review were 1 star → (9.6 + 1) ÷ 3 = 3.53, and the architect drops out of the Top 3. That is how positive and negative feedback move the home page ranking.

ArchitectInquiryServiceImpl

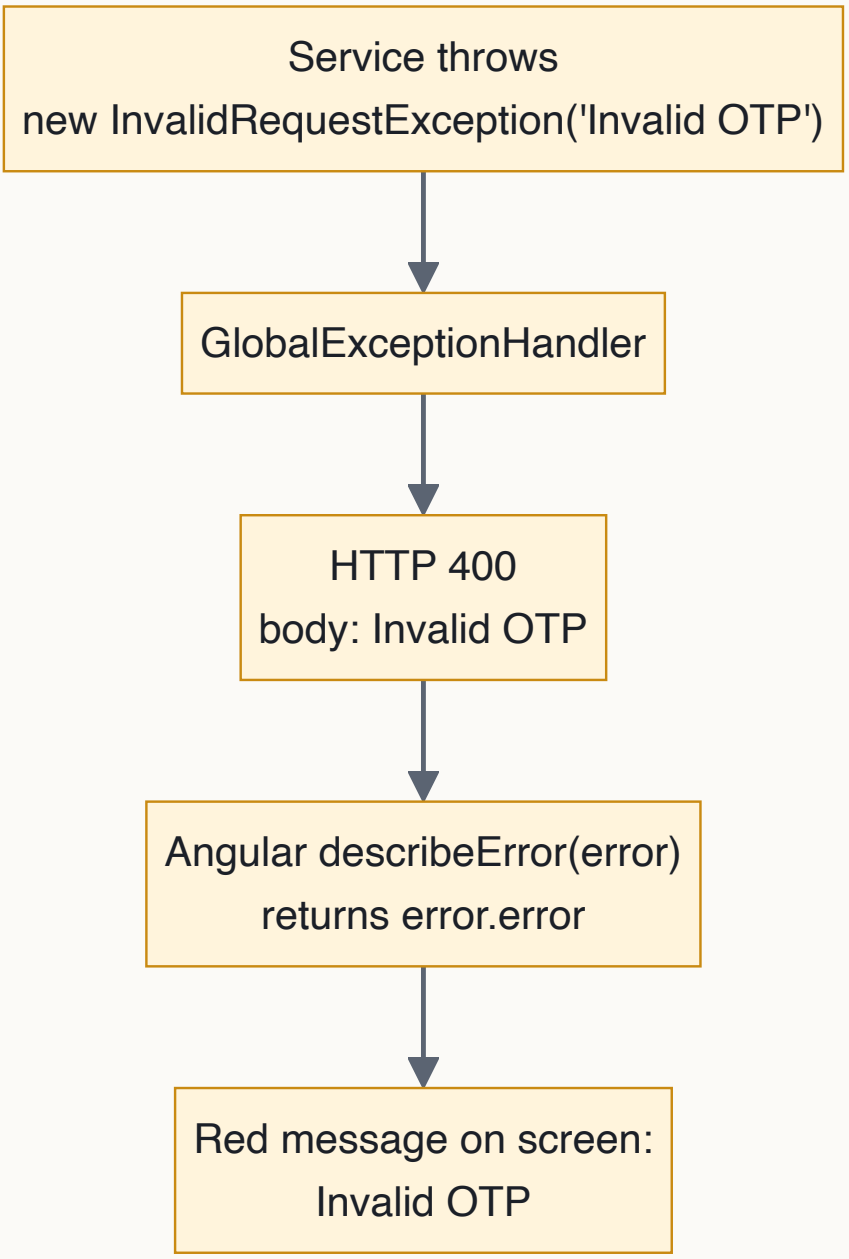
Method	Rules
createInquiry	User and architect ids required and must exist; architect must **not be Inactive**; starts with an empty follow-up list.
updateInquiry(id, data)	Admin edits message, status, priority, contact details. Follow-ups are kept.
getInquiriesByUserId / getAllInquiries	Plain reads.
deleteInquiry	Real delete (follow-ups are deleted too by cascade).
addSubInquiry(id, message, email, fromAdmin)	A user may only follow up on **their own** inquiry; message not empty; inquiry not **Closed**. Saves sender = "User" or "Admin" and the time.

FeedbackServiceImpl

Method	Rules
createFeedback	User required and must exist; architect optional but must exist if given; rating must be 1-5 if given.
getFeedbackById / getAllFeedback / getFeedbackByUserId	Reads.
deleteFeedback	Real delete.

6.5 Errors: from exception to HTTP status

Services never build HTTP responses. They throw an exception; `GlobalExceptionHandler` (`@RestControllerAdvice`) catches it everywhere and returns a status code with the message as plain text.

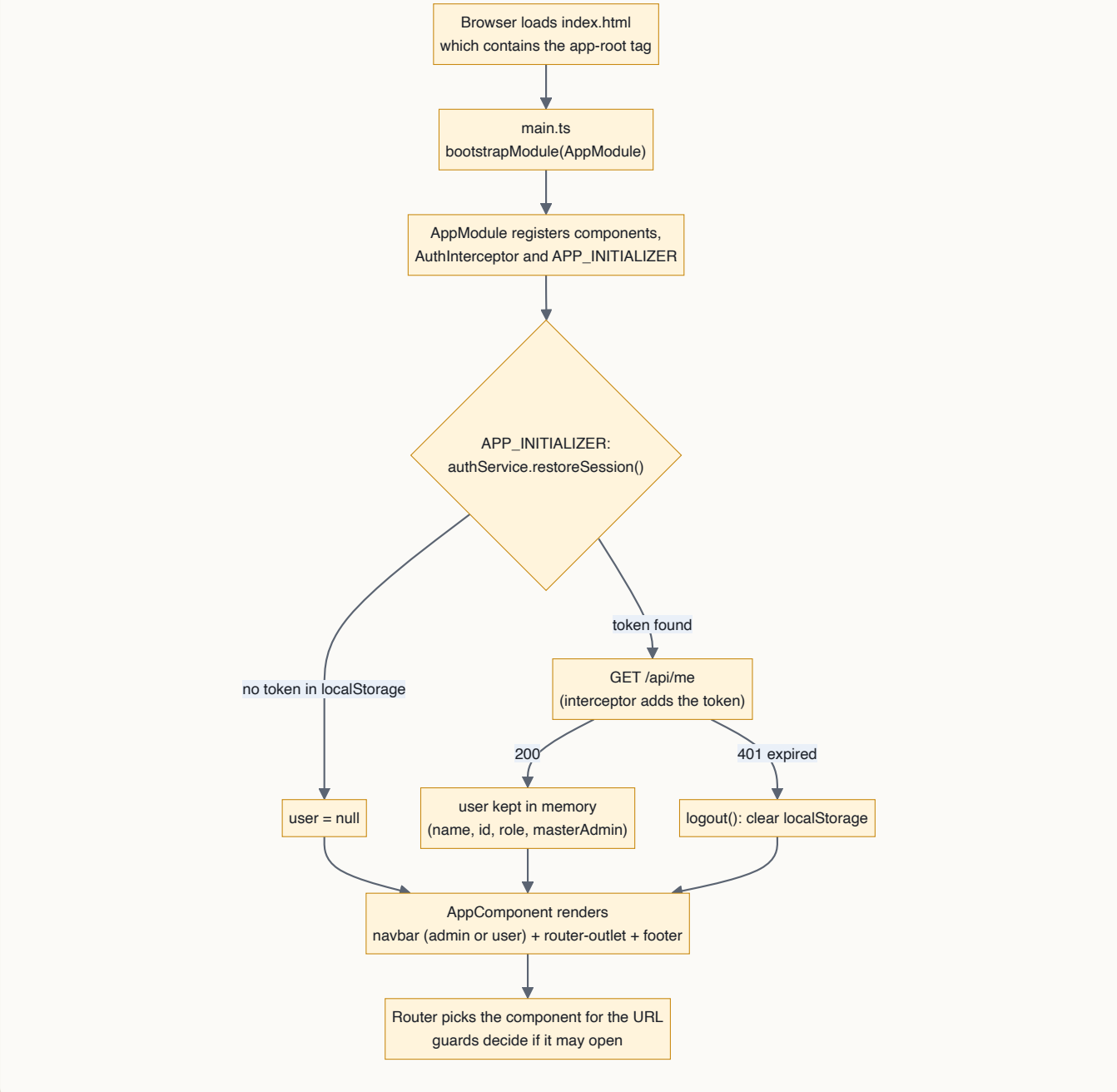


Exception	HTTP status	Typical cause
ResourceNotFoundException (and subclass ArchitectException)	404	id does not exist
InvalidRequestException (and subclass ArchitectInquiryException)	400	broken rule (inactive architect, bad OTP, rating 7...)
InvalidCredentialsException	401	wrong email or password

DuplicateArchitectException	409	duplicate record
DataIntegrityViolationException (from the database)	409	row still referenced by another table

7. Frontend core (Angular)

7.1 What happens when the page opens



APP_INITIALIZER makes Angular wait for `restoreSession()` before showing anything. That is why a page refresh keeps you logged in without storing your name or role in the browser.

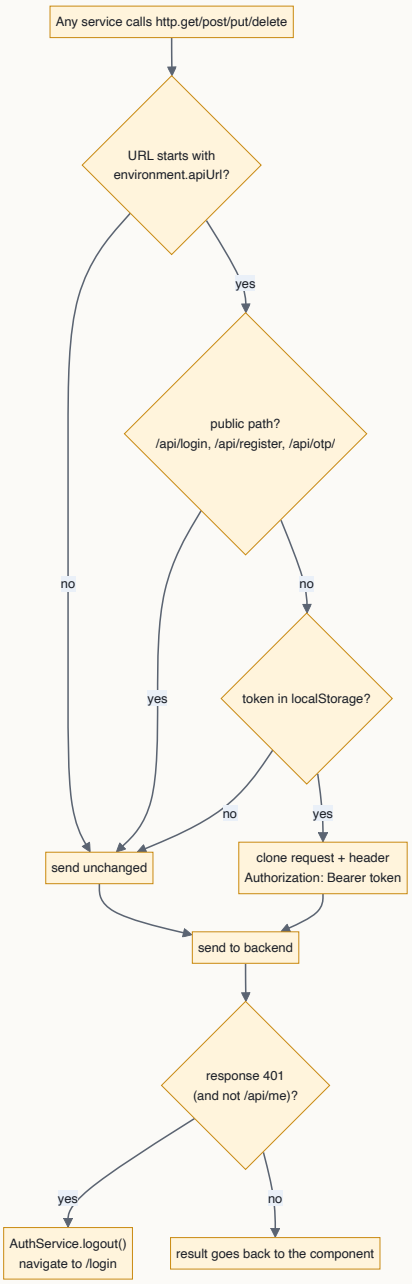
```
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
  {
    provide: APP_INITIALIZER,
    multi: true,
    deps: [AuthService],
    useFactory: (auth: AuthService) => () => auth.restoreSession()
  }
]
```

7.2 environment.ts - the backend address

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080'
};
```

Every service builds its URLs from `environment.apiUrl`, and the interceptor uses it to recognise backend calls. `environment.prod.ts` is the same file for production builds (`ng build`); `angular.json` swaps it in automatically. **To point the app at another backend, change only these two files.**

7.3 AuthInterceptor - one place that adds the token



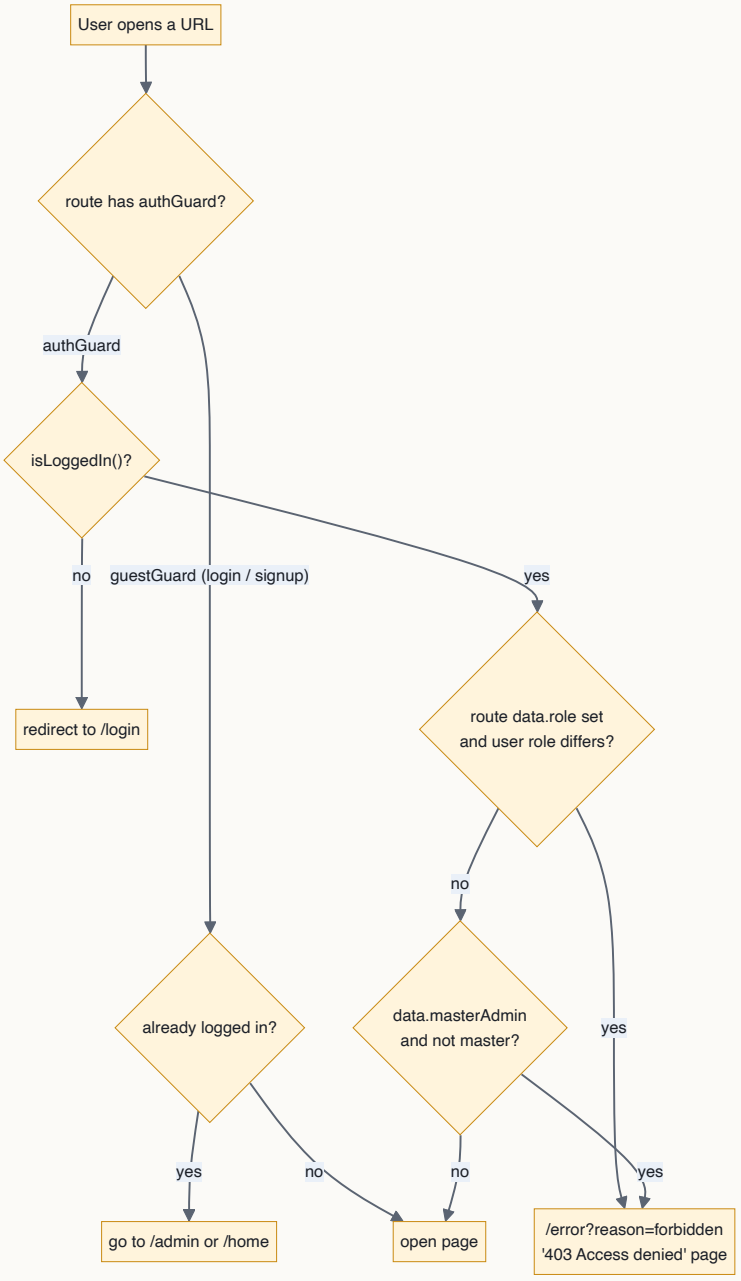
```
intercept(request: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
  const isBackend = request.url.startsWith(environment.apiUrl);
  const isPublic = AuthInterceptor.PUBLIC_PATHS.some(path => request.url.includes(path));
  const token = localStorage.getItem('token');

  const authorized = isBackend && !isPublic && token
    ? request.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : request;

  return next.handle(authorized).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401 && isBackend && !isPublic && !request.url.endsWith('/api/me')) {
        this.injector.get(AuthService).logout();
        this.router.navigate(['/login']);
      }
      return throwError(() => error);
    })
  );
}
```

- Requests are immutable in Angular, so we `clone()` with the extra header.
- `this.injector.get(AuthService)` is fetched lazily because `AuthService` itself uses `HttpClient`; injecting it directly would create a circular dependency.
- `/api/me` is excluded from the redirect because `restoreSession()` handles that failure itself during startup.
- Because of this interceptor, no service has to add headers by hand.

7.4 Guards - who may open which page



Even unknown URLs (`**`) use `authGuard`, so a wrong URL while logged out goes to the login page, and while logged in shows the "house collapsed" 404 page.

7.5 Services

Service	Methods → backend URL
AuthService	register → POST /api/register · sendOtp → POST /api/otp/send · verifyOtp → POST /api/otp/verify · login → POST /api/login (saves token, keeps user) · restoreSession → GET /api/me · changePassword → PUT /api/me/password · plus helpers getToken, getUserRole, getUserId, getUserName, isAdmin, isUser, isMasterAdmin, isLoggedIn, logout

ArchitectService	getAllArchitects, getTopRated, getArchitectById, addArchitect, updateArchitect, deleteArchitect → /api/architects...
ArchitectInquiryService	addInquiry, getAllInquiries, getInquiriesByUserId, updateInquiry, deleteInquiry, addSubInquiry → /api/inquiries...
FeedbackService	sendFeedback, getAllFeedbacksByUserId, deleteFeedback, getFeedbacks → /api/feedback...
AdminService	getUsers, makeAdmin, removeAdmin → /api/admin/users...

Every method returns an **Observable**. Nothing is sent until a component calls `.subscribe({ next, error })`; `next` runs with the data, `error` runs on failure (and the component shows `describeError(error, '...')`).

What is stored in the browser

Where	What
localStorage	**Only** `token`. Login clears everything else first.
Memory (AuthService.currentUser)	userId, username, userRole, masterAdmin - from the login answer or from /api/me after a refresh.
Nothing stored	Theme: starts from the computer's light/dark setting; the toggle changes it until the page is reloaded.

7.6 Models (TypeScript interfaces)

Interfaces only describe the JSON shape so the editor can check our code. `User`, `Login`, `Architect`, `TopArchitect`, `ArchitectInquiry`, `SubInquiry`, `Feedback` mirror the backend classes. **TEST RULE** `ArchitectInquiry` must keep `status`, `inquiryDate` and `responseDate`; the Karma tests check them.

7.7 Routes

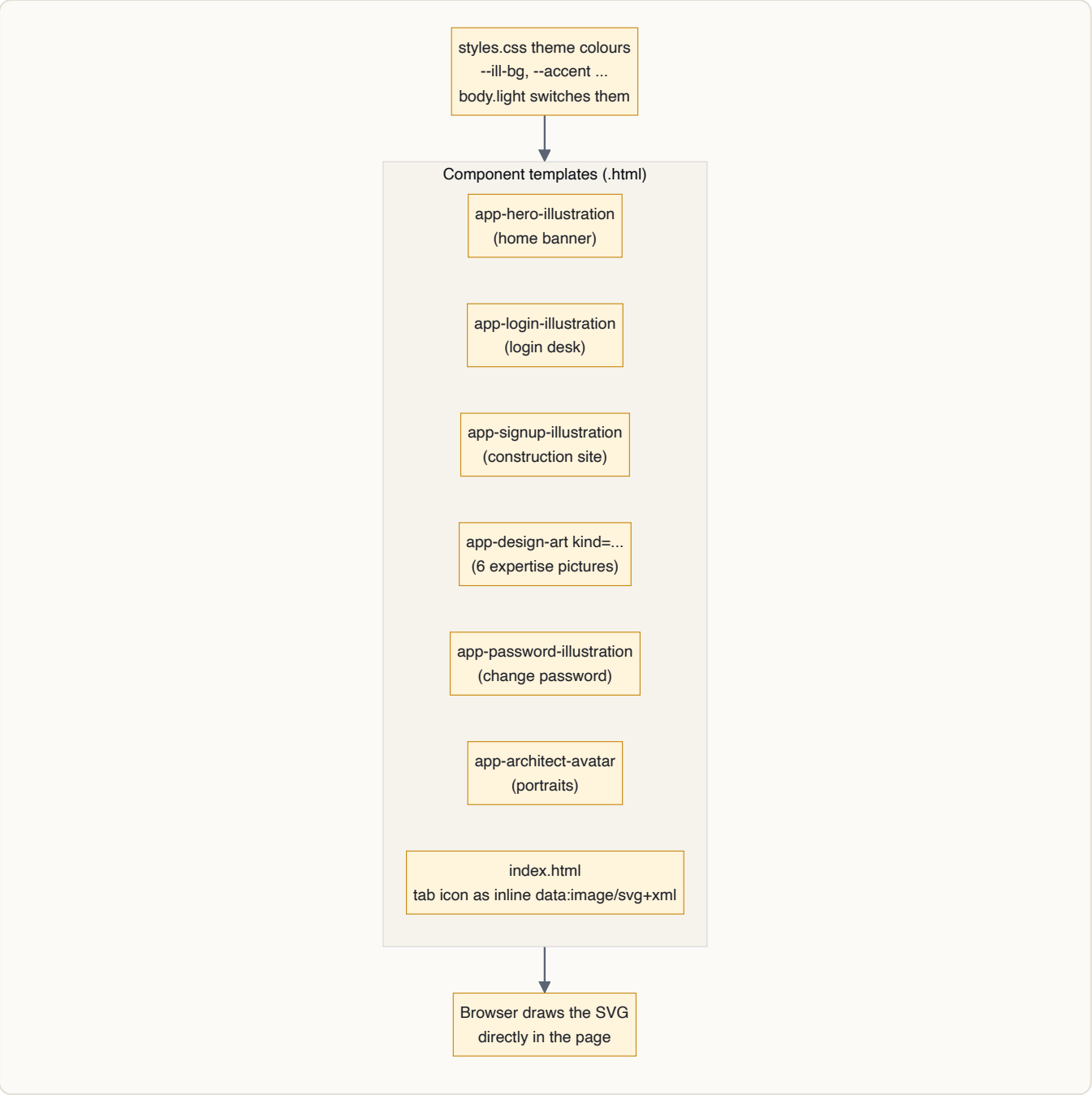
URL	Component	Guard / role
/ → /login	-	redirect
/login, /signup	LoginComponent, SignupComponent	guestGuard
/home	HomePageComponent	logged in
/portfolio/:id	ArchitectPortfolioComponent	logged in
/change-password	ChangePasswordComponent	logged in
/admin	AdminControlPanelComponent	Admin
/admin/architects, /admin/architects/add, /admin/architects/edit/:id	AdminViewArchitect, AdminAddArchitect, AdminEditArchitect	Admin
/admin/inquiries, /admin/feedback	AdminViewInquiry, AdminViewFeedback	Admin
/admin/users	AdminManageUsersComponent	Admin + masterAdmin

/user/architects	UserViewArchitectComponent	User
/user/inquiry?architectId=…	UserAddInquiryComponent	User
/user/inquiries	UserViewInquiriesComponent	User
/user/feedback/add?architectId=…	UserAddFeedbackComponent	User
/user/feedback	UserViewFeedbackComponent	User
/error, ** (anything else)	ErrorComponent	logged in

`scrollPositionRestoration: 'top'` + CSS `scroll-behavior: smooth` make every page change glide back to the top.

7.8 Where the pictures come from

The project contains **no image files**. Every picture is SVG drawing code inside an Angular component template, so it needs no file, no link and no network.



- **Theme-aware:** the drawings use CSS colour variables, so they change with light/dark mode.
- **design-art** contains six `<svg>` drawings and shows one with `*ngSwitchCase` depending on `kind` (residential, commercial, interior, landscape, urban, sustainable). Internal ids are prefixed (`residential-bg`) so several on one page don't clash.
- **architect-avatar** draws a portrait from the architect's id (`seed`): background, skin, hair or helmet, glasses and shirt are picked with simple maths, so the same architect always gets the same face.
- To check: search the project for `.png/.jpg/.svg` files (none), or open DevTools → Network (no image requests).

7.9 Styling and themes

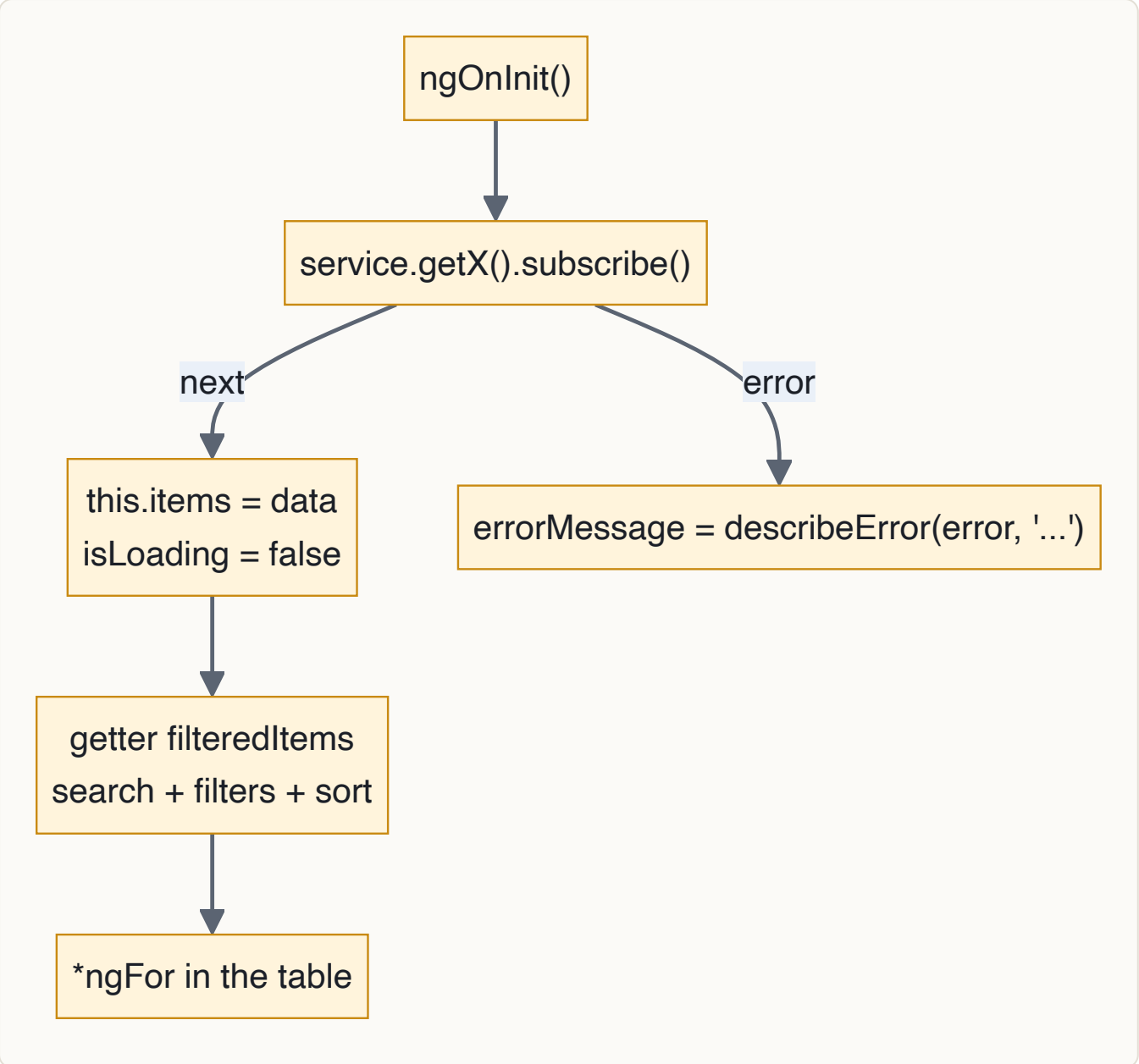
- `src/styles.css` is the design system: colour tokens (`--page-bg` , `--text` , `--accent` gold, `--input-bg` ...), buttons (`.btn` , `.btn-secondary` , `.btn-danger` , `.btn-accent`), forms, tables (`.data-table`),

status pills (`.status.pending`...), modals and the blueprint grid background.

- **Dark** is the default token set on `body`; **light** overrides them under `body.light`. The theme buttons just add/remove the `light` class.
- Each component's `.css` only holds layout specific to that screen.
- Fonts: Inter (text) and Space Grotesk (headings), from Google Fonts.

8. Every component, one by one

An Angular component = **.ts** (data and methods) + **.html** (what is shown, using `{{ value }}`, `*ngIf`, `*ngFor`, `(click)="method()"`, `[(ngModel)]="field"`) + **.css** (styles for this screen only). Most list screens follow the same pattern:



Many screens also share the same small **alert modal** pattern: an `alert` object `{show, type, title, message}`, `showAlert()` to open it, `closeAlert()` to close it (sometimes navigating afterwards), and `handleModalKeydown()` so Escape closes it.

8.1 App shell and navigation

Component	What it does	Key members

AppComponent (app.component)	The root. Shows `app-adminnav` for admins or `app-usernav` for users (only when logged in), the `<router-outlet>` where pages appear, and the footer. Starts the theme from the computer's light/dark setting.	isLoggedIn, isAdmin
AdminnavComponent	Admin top bar: Dashboard, Architects (dropdown: Add / Manage), Inquiries, Feedback, and **Admins** only for the Master Admin. Avatar button opens the account menu: name + role, Change password, Light/Dark mode, Logout (with confirm modal). Mobile hamburger menu.	isMenuOpen, isProfileOpen, isSubMenuOpen, roleLabel, isMasterAdmin, closeProfileOnOutsideClick (@HostListener document click), toggleTheme, askLogout / confirmLogout
UsernavComponent	User top bar: Home, Architects, My Inquiries, Feedback + the same account menu.	same as admin nav without the Admins link
SiteFooterComponent	Footer on every logged-in page: brand, links for the role, team "Spring Warriors" with leader card, Back to top.	year, teamName, leader, members, isAdmin, scrollTop()
ErrorComponent	Two cases: **403 Access denied** (from the guard, `?reason=forbidden&need=Admin`) with "Go to my home" / "Sign in with another account"; or **404** with the animated collapsing-house drawing.	isForbidden, neededRole, currentRole, homeLink, switchAccount(), goBack()

8.2 Login, signup, password

Component	What it does	Key members
LoginComponent	Email + password form on a card, with the desk illustration. On success goes to `/admin` (admin) or `/home` (user). Shows "Invalid Email or Password" or "Unable to connect".	loginData {email, password}, emailPattern, login(), toggleTheme()
SignupComponent	Username, Email (+ OTP), Password, Confirm, Mobile. Role is always **User**. Register is enabled only when the form is valid, passwords match and `emailVerified` is true. Success modal then goes to login.	user, confirmPassword, emailVerified, register(), closeModal()
OtpVerificationComponent	Send OTP / code box / Verify / resend countdown. Fully explained in 5.7.	@Input email, emailValid · @Output verified · sendOtp, verifyOtp, startTimer
ChangePasswordComponent	Current / new / confirm password with "Show passwords". On success shows an animated tick and "Password updated".	passwordsMatch, save() → AuthService.changePassword, isDone, homeLink

8.3 User screens

Component	What it does	Key members
HomePageComponent (/home)	Hero banner, How it works (4 steps), **Top rated architects** (live from /api/architects/top-rated, with avatar, score and review count), Explore by expertise (6 pictures), Architecture in the news (6 real events), floating testimonials (sample), call to action.	steps, expertise, news, testimonials (fixed data) · topArchitects, architectCount, cityCount, averageRating (live) · architectsRoute
UserViewArchitectComponent (/user/architects)	Architect cards with search + expertise + location filters. "View portfolio" link. **Send Inquiry** button, disabled and labelled "Currently unavailable" for Inactive architects.	filteredArchitects (getter), expertiseOptions, locationOptions, sendInquiry(id) → /user/inquiry?architectId=
ArchitectPortfolioComponent (/portfolio/:id)	Avatar, name, expertise, city, status, rating, years of practice, projects completed, about text and 4 sample projects with pictures. Users also see Send Inquiry.	buildProjects() (titles per expertise), yearsOfPractice and projectsCompleted (calculated from the id so they never change), sendInquiry(), goBack()
UserAddInquiryComponent (/user/inquiry)	Reads `architectId` from the URL, finds that architect in the list (no dropdown) and shows a "Sending to" box. Inactive → message instead of form. Form: message (≥10 chars), priority, 10-digit phone.	ngOnInit (redirects if no id), loadArchitect(), save() → addInquiry with status "Pending"
UserViewInquiriesComponent (/user/inquiries)	Table of my inquiries. Under each: the follow-up thread (Admin replies in blue, "You" in gold) and a follow-up box while not Closed; an "on hold" warning if the architect became Inactive; when Closed, a **Give feedback** button or "You rated ★★★★★". Withdraw with confirm modal.	loadInquiries, loadMyRatings, ratingFor, giveFeedback, drafts (text per inquiry), sendSubInquiry, replaceInquiry, confirmDelete
UserAddFeedbackComponent (/user/feedback/add)	Opened from a closed inquiry with `?architectId=` (no dropdown). 5-star picker with labels Poor / Fair / Good / Very good / Excellent (hover preview), text, category, date.	rating, hoverRating, shownRating, ratingLabels, save() → sendFeedback
UserViewFeedbackComponent (/user/feedback)	My feedback with stars, search, category filter, delete. Empty state explains that feedback is given from closed inquiries.	filteredFeedbacks, architectName, confirmDelete

8.4 Admin screens

Component	What it does	Key members

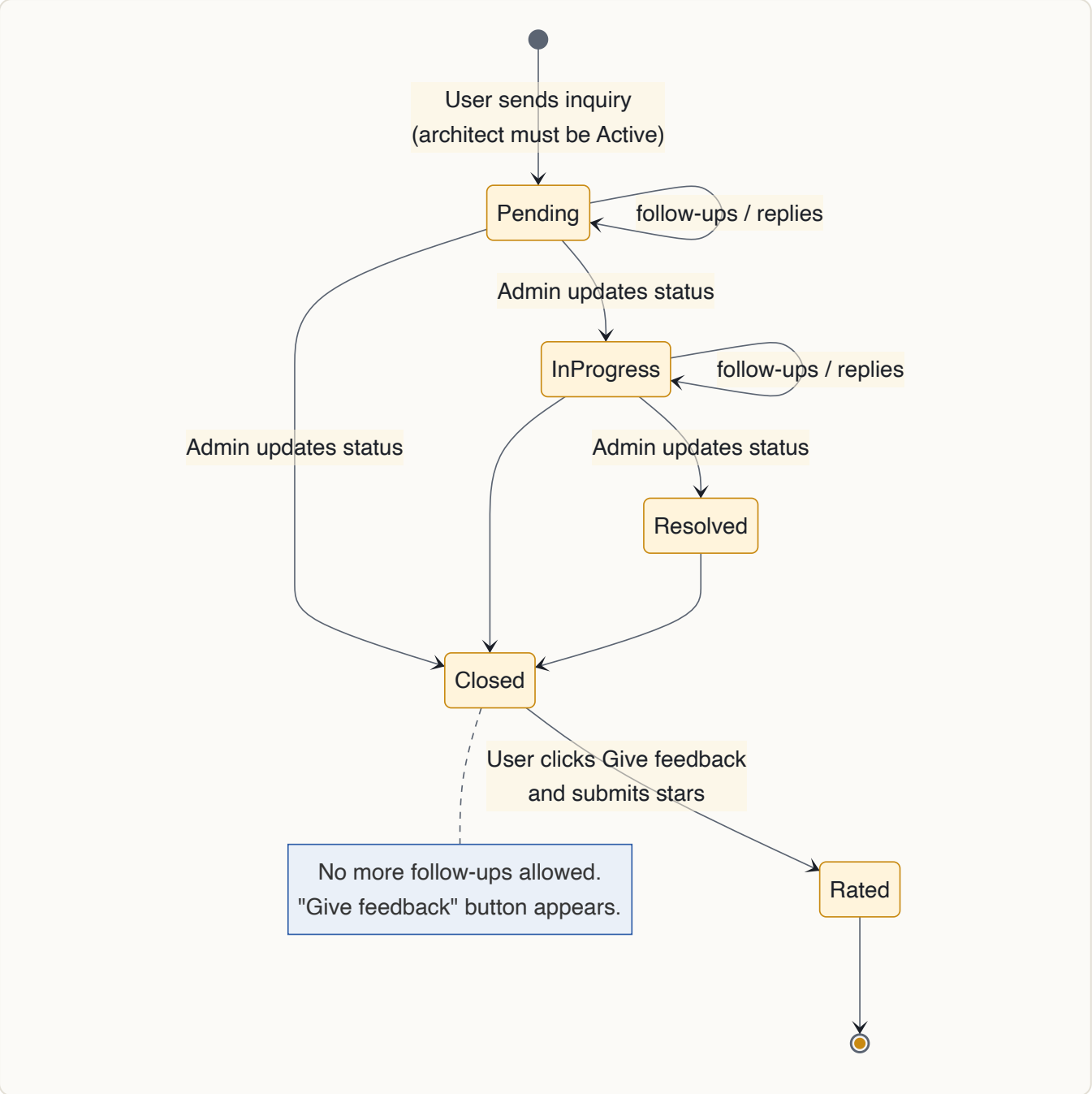
AdminControlPanelComponent (/admin)	4 live stat cards (architects, inquiries, feedback, average rating) + **analytics with sample data**: area/line chart (monthly inquiries), donut (inquiries by status), pie (architects by expertise), semicircle gauge (satisfaction), column chart (ratings). Quick-action tiles.	inquiryStatus, expertiseShare, monthlyInquiries, satisfaction, ratingBars (sample data) · segments() builds donut slices with stroke-dasharray · pieArcs() builds real SVG arc paths · linePoints/linePath/areaPath for the line chart
BarChartComponent	Reusable bar/column chart used for ratings.	@Input heading, subtitle, bars, unit(s), orientation · max, total, size(), share()
AdminViewArchitectComponent (/admin/architects)	Table of architects: admin rating plus "Users: ★ avg (count)" from feedback. Search, expertise and status filters, and **Sort**: user rating high/low, rating high/low, active first / inactive first. Update and Delete (soft delete, with explanation in the modal).	loadUserRatings() (averages feedback stars per architect), userAverage(), filteredArchitects with a `sorters` map, confirmDelete()
AdminAddArchitectComponent / **AdminEditArchitectComponent**	Form for name, expertise, location, rating, portfolio link (must start with http/https), status. Edit loads the architect by id first.	architect, expertiseOptions, statusOptions, urlPattern, save()
AdminViewInquiryComponent (/admin/inquiries)	All inquiries with search + status + priority filters. "Update Status" modal (Pending / In Progress / Resolved / Closed). Under each inquiry: the thread and a **reply box** while not Closed. Tags architects that became inactive.	openStatusModal, confirmStatusUpdate (PUT the whole inquiry with the new status), drafts, sendReply()
AdminViewFeedbackComponent (/admin/feedback)	All feedback with user, architect, stars, category, date. Search, category filter, sort by rating high/low.	filteredFeedbacks, sortBy, userName, architectName
AdminManageUsersComponent (/admin/users, Master Admin only)	All users with role pill (User / Admin / Master Admin). **Make admin** for users, **Remove admin** for admins (confirm modal). The master admin row is marked Protected.	loadUsers, makeAdmin, canRemove, askRemove / confirmRemove, roleLabel

8.5 Drawing components (no logic)

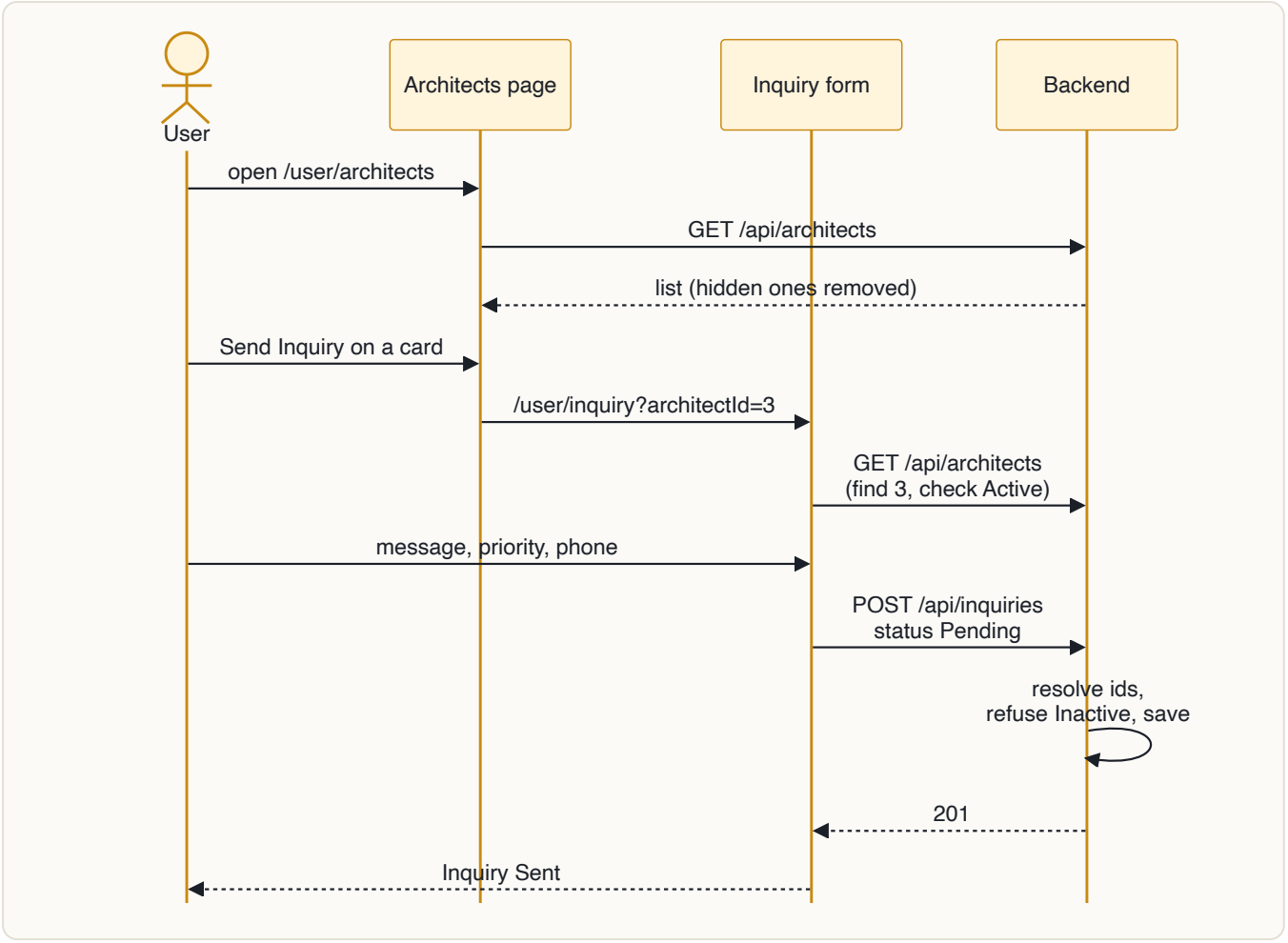
Component	Shows
HeroIllustrationComponent	House turning from blueprint into a finished building (home banner). Own colours in `hero-scene.css`, switches with `:host-context(body.light)`.
LoginIllustrationComponent	Architect's desk with blueprint and lamp.
SignupIllustrationComponent	Construction site with a crane.
PasswordIllustrationComponent	House guarded by a padlock shield, floating key and password field.
DesignArtComponent	One of six expertise pictures, chosen by `@Input() kind`.
ArchitectAvatarComponent	Illustrated portrait from `@Input() seed / size / name`.

9. End-to-end flows

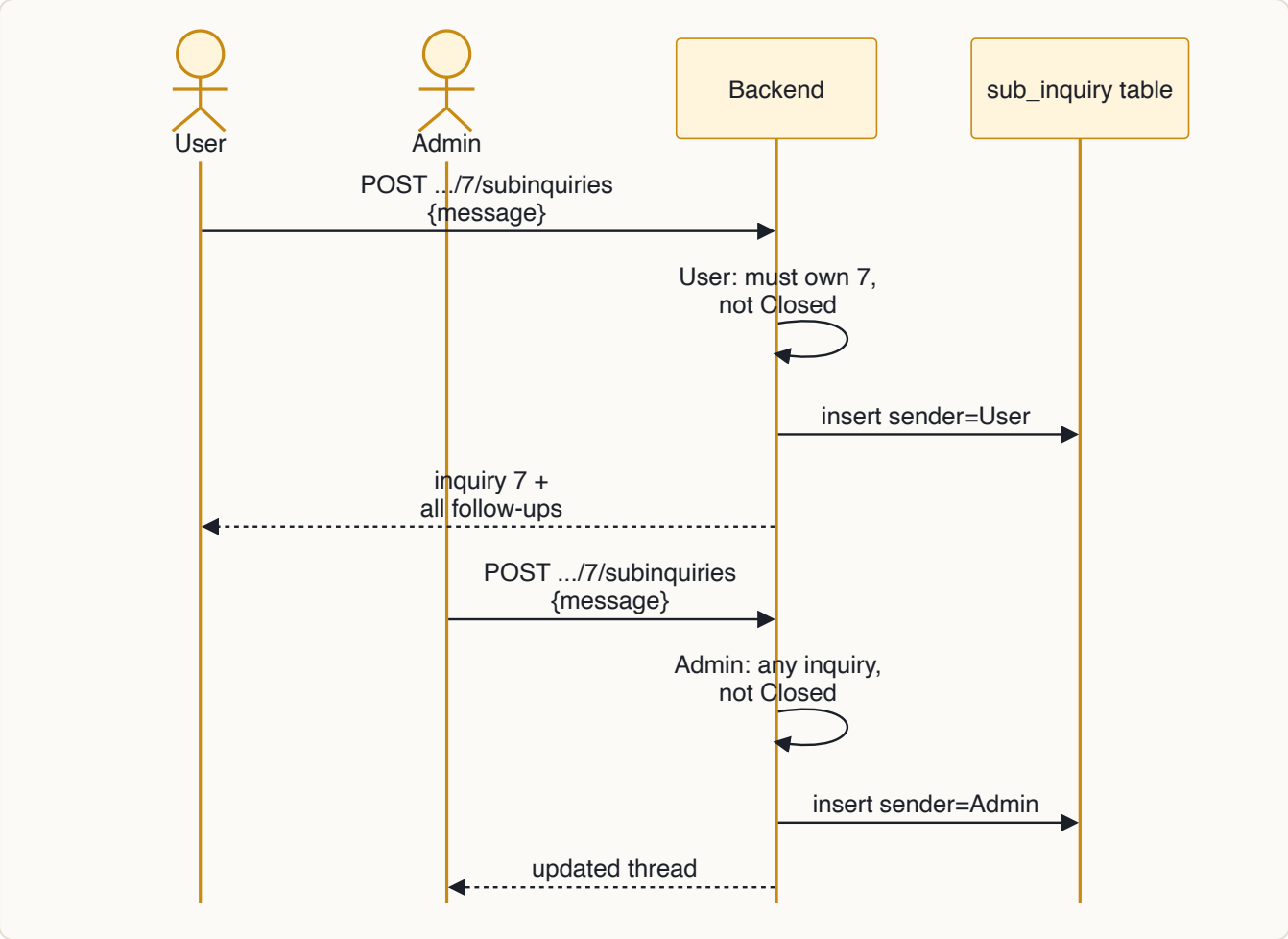
9.1 The life of an inquiry



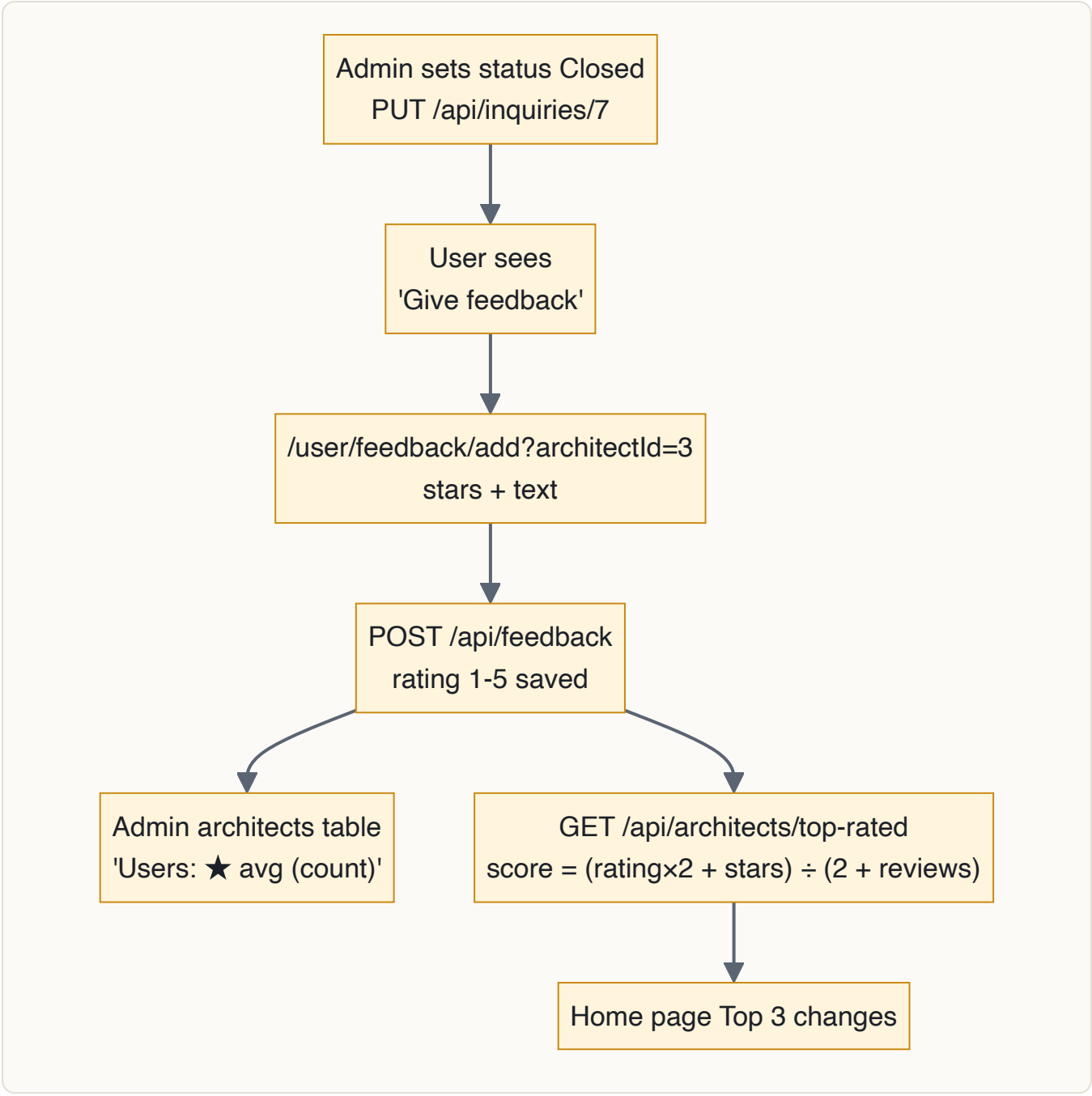
9.2 Browse → send inquiry



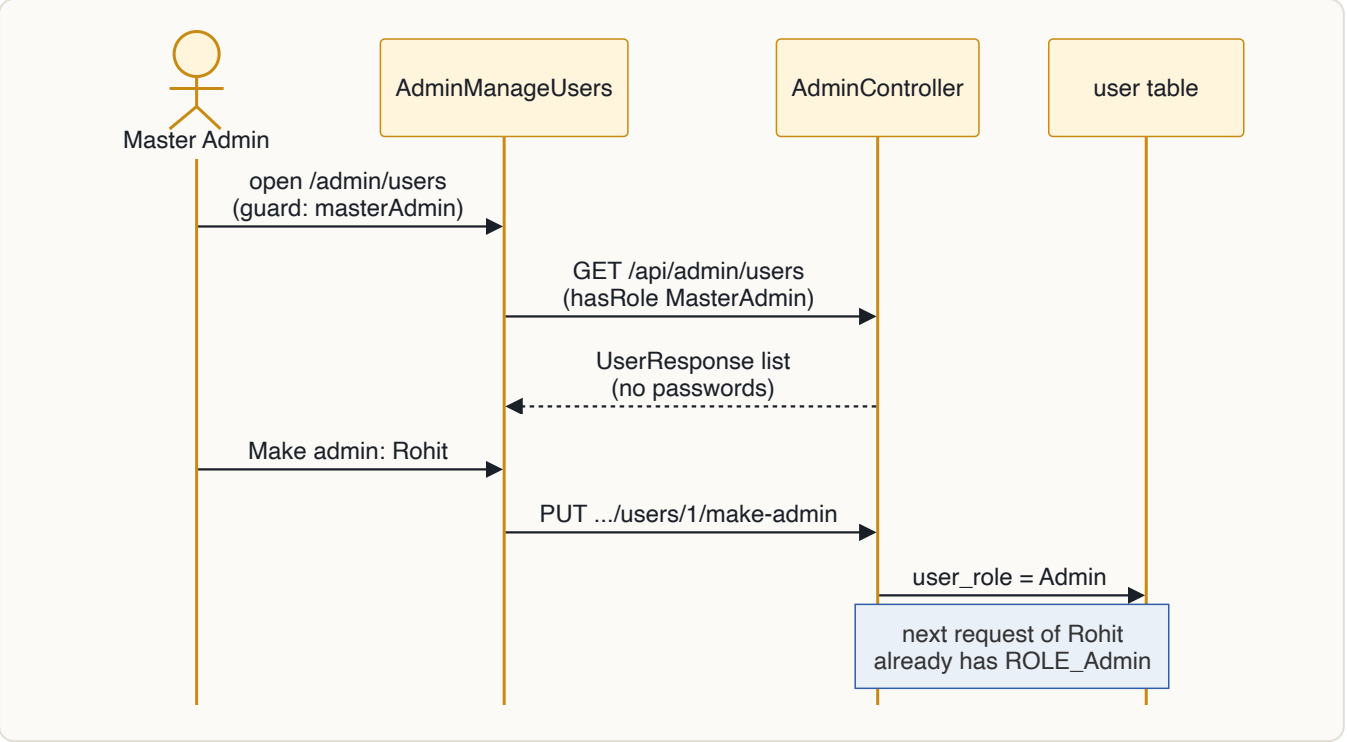
9.3 Follow-up conversation



9.4 Close → feedback → ranking



9.5 Master admin promotes a user

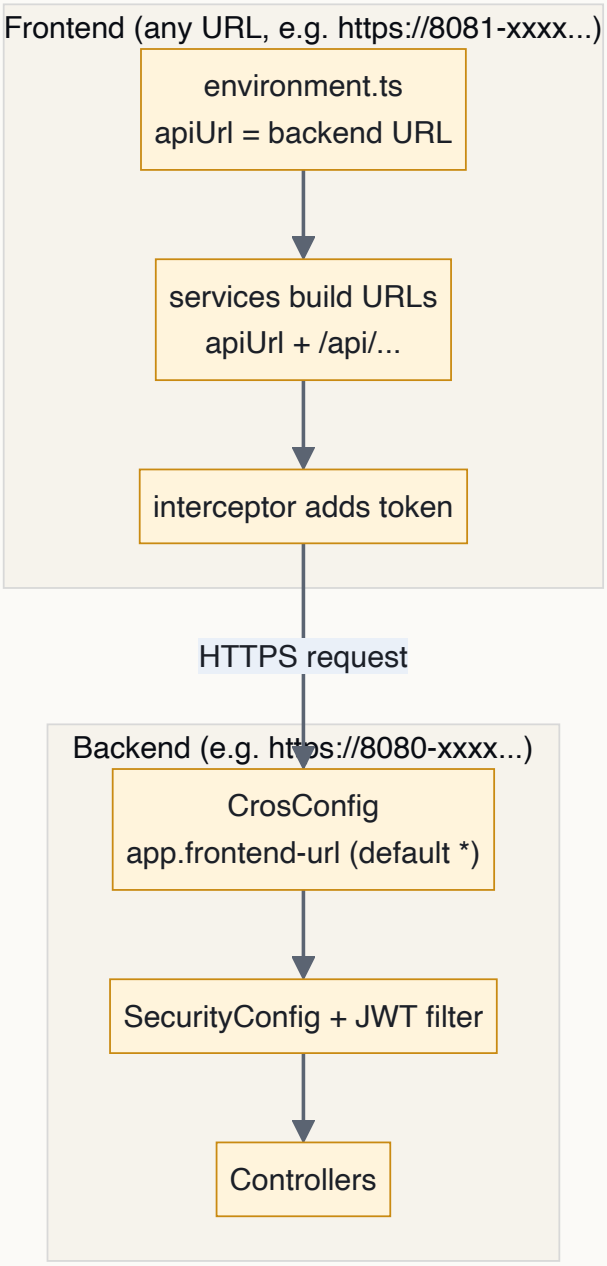


9.6 Soft delete of an architect

Admin clicks Delete → `DELETE /api/architects/5` → service sets `deleted = true` → `getAllArchitects()` uses `findByDeletedFalse()`, so it disappears from every list, but old inquiries and feedback still point to row 5 and still show the name.

10. Integration, running and testing

10.1 How frontend and backend are connected



Only **two things** connect the two sides:

1. **Frontend → backend address:** `angularapp/src/environments/environment.ts` and `environment.prod.ts` → `apiUrl`. Use the backend address (port 8080), no slash at the end.
2. **Backend → which frontends may call it (CORS):** `app.frontend-url` in `application.properties`. The default `*` accepts any frontend, so normally you change nothing.

Step by step on a new machine / IDE workspace

#	Do this
1	Start MySQL. Database password must match `spring.datasource.password` (or pass `-Dspring.datasource.password=...`).
2	Copy `springapp/config/application.properties` (Gmail keys) or set `MAIL_HOST`, `MAIL_USERNAME`, `MAIL_PASSWORD`.
3	Start the backend: `cd springapp && mvn spring-boot:run` → "Started SpringappApplication", port 8080.
4	Copy the backend's public URL from the IDE (e.g. `https://8080-abc.premiumproject.examly.io`) into `apiUrl` in both environment files.
5	Start the frontend: `cd angularapp && npm install && npm start` → port 8081.
6	Open the frontend URL, sign up (OTP email arrives), or log in as the master admin.

10.2 Logins

Account	Email	Password
Master Admin	admin@architectpoint.com	Admin@123
Sample admin	seed.admin@example.com	Seed@1234
Sample user	seed.user@example.com	Seed@1234

Sample accounts exist only in the local database where they were created.

10.3 Running the graded tests

```
mysql -u root -p -e "DROP DATABASE IF EXISTS appdb; CREATE DATABASE appdb;"
cd springapp && mvn -q test            # 15 JUnit tests

cd angularapp && npm test              # 9 Karma tests
```

Rule	Why
Reset `appdb` before backend tests	Tests expect user ids 1 and 2, architect 1, inquiry 1.
Do not edit `pom.xml` or `karma.conf.js`	The grader replaces/relies on them.
Keep `architect-inquery.service.ts` misspelled	The Karma script checks that exact file name.
Login JSON key must be exactly `token`	11 tests read `body.get("token")`.
Keep Spring Security on; URL rules only	Tests check 401/403 for every role.
Never rename existing classes, fields or URLs	They are asserted by the tests.

10.4 Troubleshooting

Symptom	Likely cause → fix

Every page shows "Unable to connect to the server"	Backend not running, or wrong `apiUrl` in environment.ts.
Browser console: "blocked by CORS policy"	`app.frontend-url` set to a different address. Use `*` or the exact frontend URL.
Logged out right after login / constant 401	Token not sent (check the interceptor is registered) or JWT expired (24 h).
403 on a page	Logged in with the wrong role for that endpoint (see the role matrix 4.5).
"Could not send the OTP right now"	No mail settings, wrong Gmail app password, or Gmail blocked. Check the backend log line "Could not email the OTP".
"Please wait N seconds..."	60-second resend cooldown - expected.
Backend tests fail from test 5 onward	Database not reset (old ids).
One Karma error breaks all 9 tests	Any TypeScript error anywhere stops compilation. Run `npx tsc -p tsconfig.app.json --noEmit`.

10.5 Glossary

Term	Simple meaning
Entity	Java class saved as a table row (`@Entity`).
Repository	Interface that reads/writes one table; Spring writes the SQL.
Service	Class with the business rules.
Controller	Class that maps URLs to Java methods.
DTO	Shape of a request/response that is not a table.
JWT	Signed text token proving who you are; sent as `Authorization: Bearer ...`.
BCrypt	One-way hashing for passwords and OTP codes.
401 / 403	401 = not logged in. 403 = logged in but not allowed.
CORS	Browser rule about which websites may call an API on another address.
OTP	One-time password: the 6-digit email code.
SMTP	The protocol used to send email.
Observable / subscribe	Angular's way of handling a result that arrives later (HTTP).
Interceptor	Code that sees every HTTP request/response (adds the token).
Guard	Code that decides whether a route may open.
Soft delete	Hide a row with a flag instead of deleting it.